SISTEMA NATYS

Informe de Pruebas de Software

Módulo de Entregas · Pruebas Unitarias y de Integración

Integrante	Elías Armas
Equipo	Equipo 8
Docente	Profa. Angelismar Terán
Sección	IN3103
Fecha	17-10-2026

1. Introducción

Este informe documenta las pruebas unitarias y de integración del modelo Entrega del Sistema Natys, encargado de registrar, consultar, modificar, eliminar y cancelar las entregas de pedidos (RF 14). Las pruebas se ejecutaron con PHPUnit directamente sobre el código del modelo, sin pasar por la interfaz.

La primera prueba (UT-01) detectó un defecto en el registro de entregas, que se corrigió y se verificó con una re-ejecución. Luego se completó la cobertura del CRUD con cuatro pruebas más (UT-02 a UT-05). Todas las pruebas unitarias se aislaron con mocks, de modo que se ejecutan sin servidor ni base de datos. Finalmente, una prueba de integración (IT-01) verificó el modelo contra la base de datos real y detectó un segundo defecto (BUG-02), también corregido.

1.1 Entorno de pruebas

Sistema operativo	Windows 11 (compilación 26200)
Servidor local	XAMPP con MySQL activo en la ejecución inicial de UT-01; apagado en todas las ejecuciones aisladas
Lenguaje	PHP 8.2.12
Herramienta	PHPUnit 11.5.57 (instalado con Composer), con mocks de PHPUnit
Configuración	phpunit.xml · bootstrap: vendor/autoload.php · carpeta de pruebas: tests
Proyecto	C:\xampp\htdocs\natys-larence

1.2 Resumen de resultados

ID	Caso de prueba	Acción	Estado
UT-01	Registrar entrega con datos inválidos (ejecución inicial)	registrar	**Fallido (defecto encontrado)**
UT-01 (R)	Re-ejecución aislada tras corregir BUG-01	registrar	**Aprobado**
UT-02	Consultar entregas devuelve listado	consultar	**Aprobado**
UT-03	Modificar entrega inexistente es rechazada	modificar	**Aprobado**
UT-04	Eliminar entrega existente	eliminar	**Aprobado**
UT-05	Cancelar entrega sin motivo es rechazada	cancelar	**Aprobado**
Suite	Archivo completo tests/EntregaTest.php (7 pruebas)	todas	**7 de 7 aprobadas**
IT-01	Integración: modificar entrega actualiza repartidor (ejecución inicial)	modificar	**Fallido (defecto encontrado)**
IT-01 (R)	Re-ejecución tras corregir BUG-02	modificar	**Aprobado**
API-01	Backend: obtener entrega por ID (Postman)	obtener	**Aprobado**
SM-01	Humo: el módulo de Entregas enciende (5 verificaciones)	todas	**Aprobado**
UI-01	Interfaz y sistema: Modificar muestra los datos (Selenium IDE)	modificar	**Aprobado**
SEG-01	Seguridad: acceso sin sesión (Postman)	obtener	**Aprobado**
SEG-02	Seguridad: inyección SQL en filtro por sector (Postman)	obtener_clientes_por_sector	**Aprobado**
RD-01	Rendimiento: 50 usuarios simultáneos (JMeter)	obtener	**Aprobado**
AC-01	Aceptación: consultar y ver detalle, RF 14.2 y 14.5 (Codeception)	consultar / obtener	**Aprobado**
RG-01	Regresión final: todas las pruebas automatizadas	todas	**Aprobado**

1.3 Cobertura del CRUD

Operación	Prueba que la cubre	Comportamiento verificado
Crear	UT-01 · Registrar entrega con datos inválidos	**Rechaza datos incompletos**
Leer (listado)	UT-02 · Consultar entregas	**Devuelve el listado**
Leer (una)	Obtener entrega exitosamente con mock	**Devuelve una entrega por su id**
Actualizar	UT-03 · Modificar entrega inexistente	**Rechaza y deshace la transacción**
Eliminar	UT-04 · Eliminar entrega existente	**Desactiva y confirma la transacción**
Cancelar	UT-05 · Cancelar sin motivo	**Exige el motivo antes de consultar la base**

1.4 Resumen de hallazgos

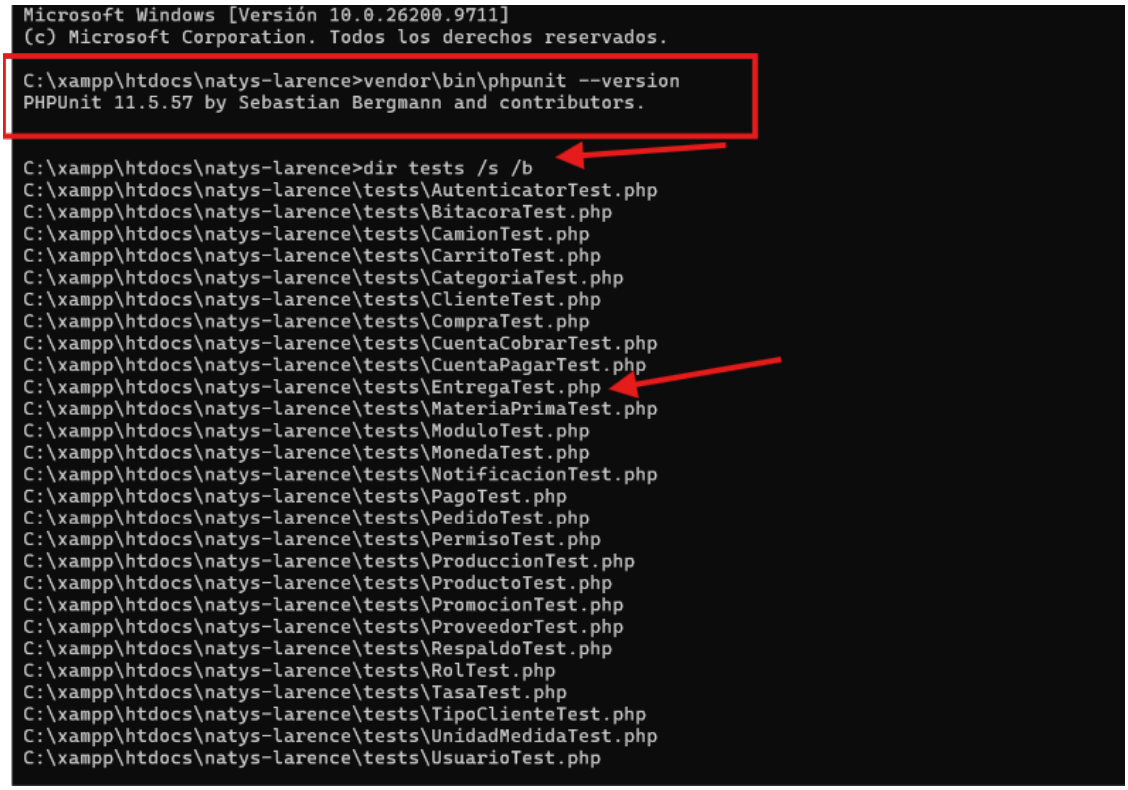
ID	Defecto	Detectado por	Estado
BUG-01	Registrar acepta datos incompletos y responde éxito ante errores de MySQL	UT-01	**Corregido**
BUG-02	Modificar usa columnas y tabla inexistentes y responde éxito sin guardar	IT-01	**Corregido**
BUG-03	Modificar borra las observaciones (sector y cajas) no enviadas	Revisión de interfaz	**Corregido**
BUG-04	El formulario de edición no carga sector, pedidos ni fecha	Revisión de interfaz	**Parcial**

2. Prueba unitaria UT-01

ID	UT-01
Nombre	Registrar entrega con datos inválidos
Tipo de prueba	Unitaria
Herramienta	PHPUnit 11.5.57
Archivo y método	tests/EntregaTest.php · test_registrar_entrega_con_datos_invalidos
Descripción	Verifica que el método registrar del modelo Entrega rechace una entrega cuando los datos obligatorios están vacíos, devolviendo status false sin enviar la inserción a la base de datos.
Precondiciones	MySQL activo en XAMPP · PHPUnit instalado en el proyecto · configuración phpunit.xml presente.
Datos de entrada	Acción: `registrar` JSON: `{"repartidor": ""}` (el resto de campos obligatorios ausentes)
Pasos	1. Se inició el servicio MySQL desde el panel de XAMPP. 2. Se abrió CMD en la carpeta C:\xampp\htdocs\natys-larence. 3. Se verificó la instalación con: vendor\bin\phpunit --version 4. Se ejecutó la prueba con: vendor\bin\phpunit tests\EntregaTest.php --testdox 5. Se registró la salida de la terminal y se revisó la tabla entregas.
Resultado esperado	El modelo devuelve `status = false` con un mensaje de validación y no intenta insertar en la tabla entregas.
Resultado obtenido	El modelo devolvió `status = true`. PHPUnit reportó: `Failed asserting that true is false` (EntregaTest.php, línea 29). El modelo no validó los datos y envió la inserción a MySQL, que la rechazó con dos errores: • 1452: falla la clave foránea `id_estado_entrega` (entregas_ibfk_1). • 1048: la columna `id_pedido` no puede ser nula. Resumen de la ejecución: 3 pruebas, 11 aserciones, 1 fallo.
Estado	**Fallido (defecto encontrado)**
Fecha de ejecución	16-10-2026
Ejecutado por	Elías Armas
Observación	En esta ejecución la prueba usaba la clase Entrega real (new Entrega()), por lo que llegó a la base de datos larence. La versión aislada se documenta en la sección 4.3.

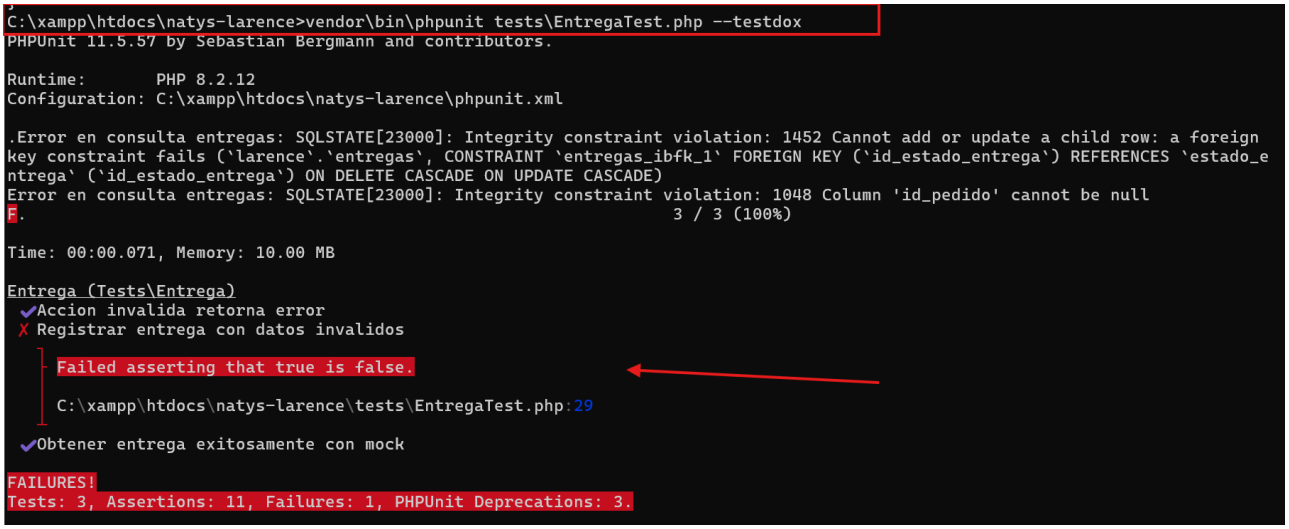
2.1 Evidencias

Captura 1 – Verificación de PHPUnit



Salida de vendor\bin\phpunit --version y contenido de la carpeta tests

Captura 2 – Resultado obtenido de la ejecución



Salida de vendor\bin\phpunit tests\EntregaTest.php --testdox antes de la corrección

3. Hallazgos

ID	BUG-01
Caso relacionado	UT-01 – Registrar entrega con datos inválidos
Descripción	El modelo Entrega no validaba los campos obligatorios antes de insertar y devolvía status true aunque MySQL rechazaba la operación (errores 1452 y 1048). El usuario recibía un mensaje de éxito sin que la entrega se guardara.
Severidad	Alta
Estado	**Corregido**

4. Corrección aplicada

4.1 Causa raíz

Causa 1	El trait ValidacionTrait solo valida los campos que llegan en el JSON. Si faltan campos obligatorios (pedido, cliente, dirección, fecha, estado, cajas), no los reporta, y el repartidor está definido como opcional para la entidad entrega.
Causa 2	Las funciones Insertar_Cabecera_Entrega e Insertar_Detalle_Entrega capturaban la excepción de PDO y devolvían un arreglo en lugar de lanzar el error. Por eso Registrar_Entrega no detectaba la falla, confirmaba la transacción y respondía «Entrega registrada con éxito».

4.2 Cambios realizados

Archivo	Función	Cambio
app/models/Entrega.php	Validar_Campos_Registro (nueva)	Verifica que pedido, cliente, dirección, fecha programada, estado y repartidor no estén vacíos, y que las cajas sean mayores a 0.
app/models/Entrega.php	manejarAccion, caso registrar	Llama a Validar_Campos_Registro antes de Registrar_Entrega; si falta un dato devuelve status false sin conectarse a la base.
app/models/Entrega.php	Insertar_Cabecera_Entrega e Insertar_Detalle_Entrega	En el bloque catch se reemplazó el return por throw $e; para que Registrar_Entrega haga rollback y devuelva status false.

Fragmento agregado en manejarAccion:

```
case 'registrar':
    $validacion = $this->setEntregaData($entrega_json);
    if (!$validacion['status']) {
        return $validacion;
    }
    // CORRECCION BUG-01
    $obligatorios = $this->Validar_Campos_Registro();
    if (!$obligatorios['status']) {
        return $obligatorios;
    }
    return $this->Registrar_Entrega();
```

4.3 Aislamiento de las pruebas unitarias

Una prueba unitaria debe evaluar solo el código del modelo, sin depender del servidor ni de la base de datos. Para lograrlo se usaron mocks (objetos dobles) de PHPUnit en todas las pruebas del archivo:

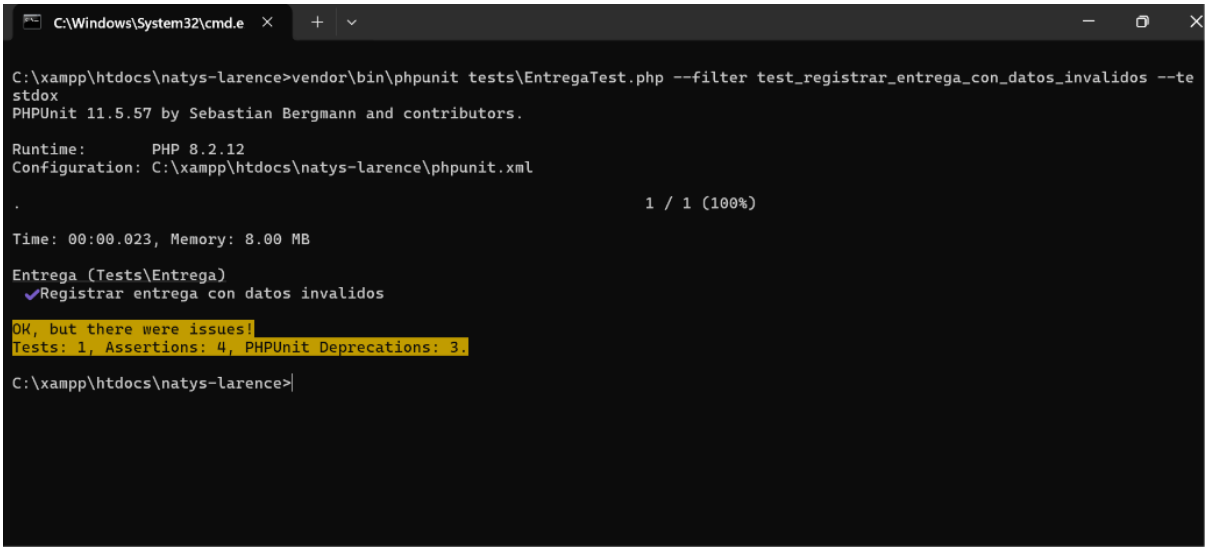
Sin constructor real	disableOriginalConstructor() evita que se ejecute el constructor de Conexion, que prepara la conexión a MySQL.
Conexión simulada	onlyMethods(['getConnectionNegocio', 'closeConnection']) reemplaza la conexión real por una falsa; cuando la prueba lo necesita, la conexión falsa devuelve una base de datos simulada (PDO y PDOStatement).
Verificaciones del mock	never() comprueba que algo no ocurra (por ejemplo, conectarse a la base o confirmar una transacción) y once() comprueba que ocurra exactamente una vez.
Pruebas existentes	Las pruebas «Acción inválida retorna error» y «Obtener entrega exitosamente con mock» se ajustaron con disableOriginalConstructor() para que tampoco dependan de MySQL.

5. Re-ejecución de UT-01 (regresión aislada)

ID	UT-01 (R)
Nombre	Registrar entrega con datos inválidos – aislada y después de la corrección
Precondiciones	XAMPP apagado (Apache y MySQL detenidos) · Entrega.php corregido · método de prueba aislado con mock.
Comando	vendor\bin\phpunit tests\EntregaTest.php --filter test_registrar_entrega_con_datos_invalidos --testdox
Resultado esperado	La prueba pasa sin servidor ni base de datos: el modelo devuelve status false con el mensaje «El pedido es requerido.» y nunca intenta conectarse a MySQL.
Resultado obtenido	✔️ Registrar entrega con datos invalidos. 1 prueba, 4 aserciones, sin fallos (0,023 s; 8 MB). El modelo respondió `status = false` y la verificación `never()` confirmó que no hubo conexión a la base de datos, aun con XAMPP apagado.
Estado	**Aprobado**
Fecha de ejecución	16-10-2026
Ejecutado por	Elías Armas

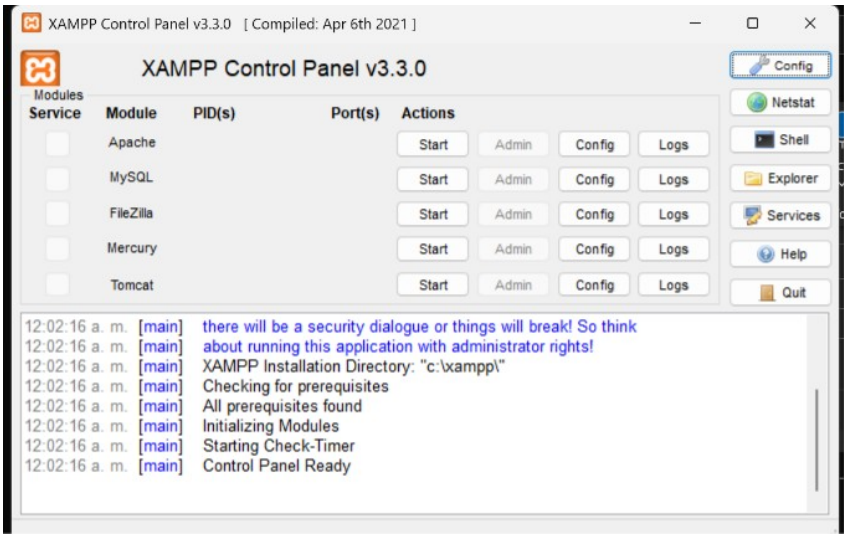
5.1 Evidencias

Captura 3 – Re-ejecución aislada de UT-01



Prueba aprobada

Captura 4 – XAMPP apagado durante la ejecución



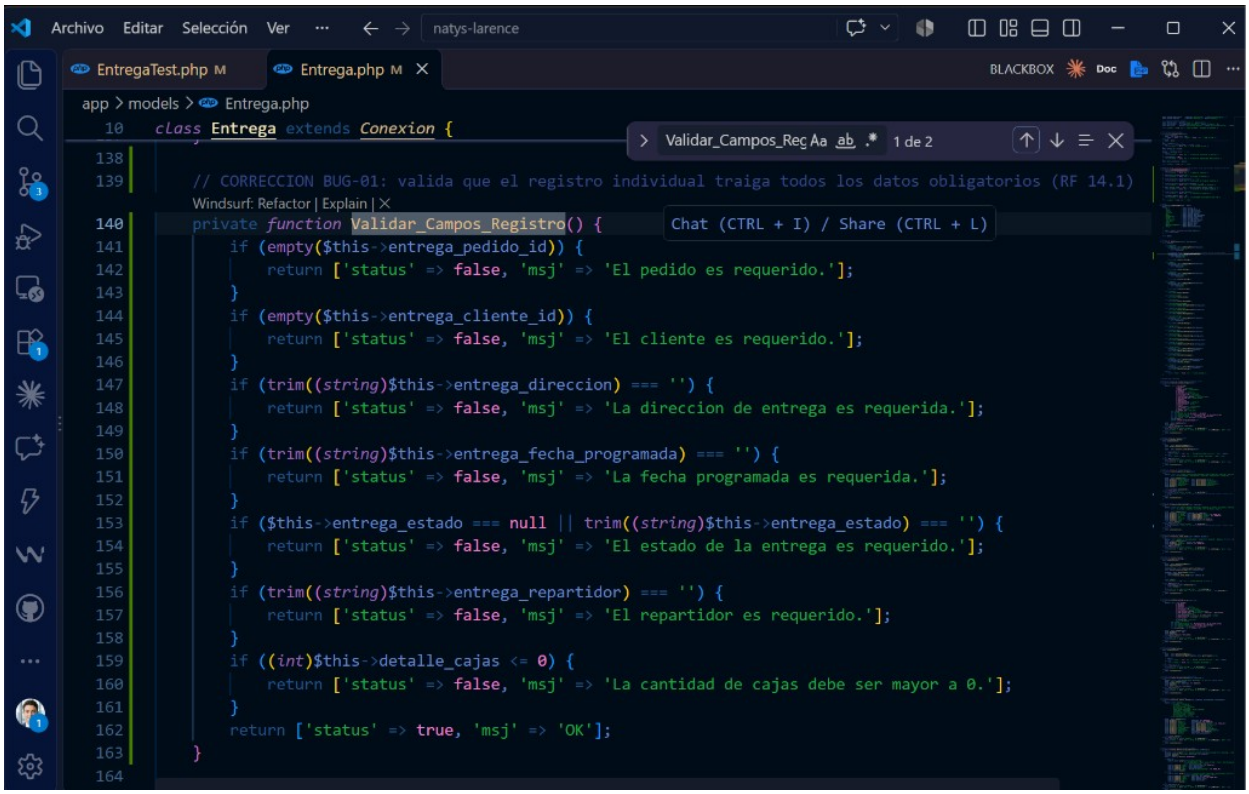
Apache y MySQL detenidos

Captura 5 – Método de prueba aislado



test_registrar_entrega_con_datos_invalidos en tests/EntregaTest.php

Captura 5b – Código corregido del modelo



Función Validar_Campos_Registro en app/models/Entrega.php

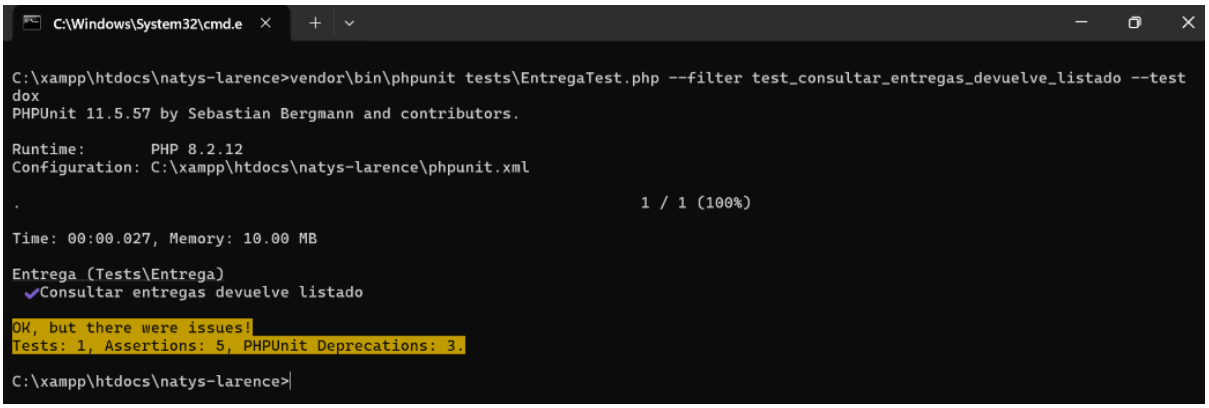
6. Prueba unitaria UT-02: Consultar entregas devuelve listado

ID	UT-02
Nombre	Consultar entregas devuelve listado
Tipo de prueba	Unitaria (aislada con mocks)
Acción evaluada	consultar
Archivo y método	tests/EntregaTest.php · test_consultar_entregas_devuelve_listado
Descripción	Verifica que la acción consultar devuelva el listado de entregas activas con status true.
Precondiciones	XAMPP apagado (Apache y MySQL detenidos) · Entrega.php corregido.
Simulación (mock)	Base de datos simulada que, al consultar, devuelve dos entregas de ejemplo. El doble de Entrega debe pedir la conexión exactamente una vez.
Datos de entrada	`Acción: consultar · JSON: {}`
Pasos	1. Se agregó el método de prueba al archivo tests/EntregaTest.php. 2. Se detuvieron Apache y MySQL en XAMPP. 3. Se ejecutó: vendor\bin\phpunit tests\EntregaTest.php --filter test_consultar_entregas_devuelve_listado --testdox 4. Se registró la salida de la terminal.
Resultado esperado	status true, mensaje «Entregas encontradas con exito.» y una lista con las 2 entregas simuladas.
Resultado obtenido	✔️ Consultar entregas devuelve listado. 1 prueba, 5 aserciones, sin fallos (0,027 s; 10 MB). El modelo devolvió las 2 entregas simuladas con el mensaje esperado y pidió la conexión una sola vez.
Estado	**Aprobado**
Fecha de ejecución	17-10-2026
Ejecutado por	Elías Armas

Código de la prueba:

```
$stmtMock = $this->createMock(PDOStatement::class);
$stmtMock->method('execute')->willReturn(true);
$stmtMock->method('fetchAll')->willReturn([ /* 2 entregas */ ]);
$pdoMock = $this->createMock(PDO::class);
$pdoMock->method('exec')->willReturn(0);
$pdoMock->method('prepare')->willReturn($stmtMock);
$entrega = $this->getMockBuilder(Entrega::class)->disableOriginalConstructor()
    ->onlyMethods(['getConnectionNegocio', 'closeConnection'])->getMock();
$entrega->expects($this->once())->method('getConnectionNegocio')->willReturn($pdoMock);
$resultado = $entrega->manejarAccion('consultar', '{}');
$this->assertTrue($resultado['status']);
$this->assertCount(2, $resultado['data']);
```

Captura 6 – Ejecución de UT-02



Prueba aprobada con XAMPP apagado

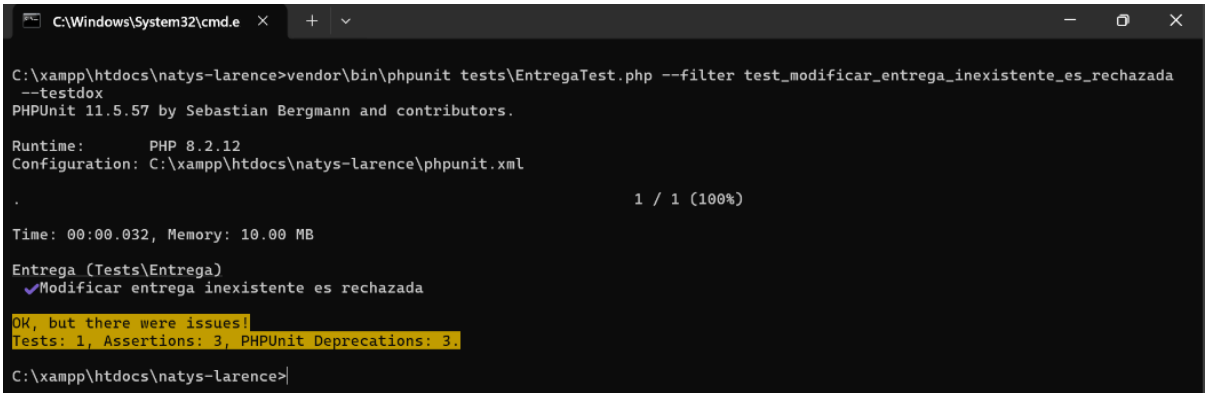
7. Prueba unitaria UT-03: Modificar entrega inexistente es rechazada

ID	UT-03
Nombre	Modificar entrega inexistente es rechazada
Tipo de prueba	Unitaria (aislada con mocks)
Acción evaluada	modificar
Archivo y método	tests/EntregaTest.php · test_modificar_entrega_inexistente_es_rechazada
Descripción	Verifica que al intentar modificar una entrega que no existe, el modelo la rechace, deshaga la transacción y no guarde cambios.
Precondiciones	XAMPP apagado (Apache y MySQL detenidos) · Entrega.php corregido.
Simulación (mock)	Base de datos simulada que no encuentra la entrega 999. Se verifica que se ejecute rollBack una vez y que commit nunca se ejecute.
Datos de entrada	`Acción: modificar · id: 999 · pedido_id: 1 · cliente_id: 1 · estado: 2 · repartidor: Carlos Perez · num_cajas: 50`
Pasos	1. Se agregó el método de prueba al archivo tests/EntregaTest.php. 2. Se detuvieron Apache y MySQL en XAMPP. 3. Se ejecutó: vendor\bin\phpunit tests\EntregaTest.php --filter test_modificar_entrega_inexistente_es_rechazada --testdox 4. Se registró la salida de la terminal.
Resultado esperado	status false con el mensaje «Entrega no existe»; la transacción se deshace (rollBack) y no se confirma (commit).
Resultado obtenido	✔️ Modificar entrega inexistente es rechazada. 1 prueba, 3 aserciones, sin fallos (0,032 s; 10 MB). El modelo rechazó la modificación, ejecutó rollBack y no ejecutó commit.
Estado	**Aprobado**
Fecha de ejecución	17-10-2026
Ejecutado por	Elías Armas

Código de la prueba:

```
$stmtMock->method('fetch')->willReturn(false);  // la entrega 999 no existe
$pdoMock->expects($this->once())->method('rollBack')->willReturn(true);
$pdoMock->expects($this->never())->method('commit');
$resultado = $entrega->manejarAccion('modificar', json_encode([ 'id' => 999, ... ]));
$this->assertFalse($resultado['status']);
$this->assertEquals('Entrega no existe', $resultado['msj']);
```

Captura 7 – Ejecución de UT-03



Prueba aprobada con XAMPP apagado

8. Prueba unitaria UT-04: Eliminar entrega existente

ID	UT-04
Nombre	Eliminar entrega existente
Tipo de prueba	Unitaria (aislada con mocks)
Acción evaluada	eliminar
Archivo y método	tests/EntregaTest.php · test_eliminar_entrega_existente
Descripción	Verifica que al eliminar una entrega existente el modelo la desactive, confirme la transacción y devuelva los datos anteriores para la bitácora.
Precondiciones	XAMPP apagado (Apache y MySQL detenidos) · Entrega.php corregido.
Simulación (mock)	Base de datos simulada donde la entrega 5 existe. Se verifica que commit se ejecute una vez y que rollBack nunca se ejecute.
Datos de entrada	`Acción: eliminar · JSON: {"id": 5}`
Pasos	1. Se agregó el método de prueba al archivo tests/EntregaTest.php. 2. Se detuvieron Apache y MySQL en XAMPP. 3. Se ejecutó la suite completa: vendor\bin\phpunit tests\EntregaTest.php --testdox 4. Se registró la salida de la terminal.
Resultado esperado	status true, mensaje «Entrega eliminada con éxito.» y data_bitacora con la entrega 5.
Resultado obtenido	✔️ Eliminar entrega existente (ejecutada dentro de la suite completa). El modelo confirmó la transacción una vez, no la deshizo y devolvió los datos de la entrega 5 para la bitácora.
Estado	**Aprobado**
Fecha de ejecución	17-10-2026
Ejecutado por	Elías Armas

Código de la prueba:

```
$stmtMock->method('fetch')->willReturn(['id_entregas' => 5, 'repartidor' => 'Carlos Perez']);
$pdoMock->expects($this->once())->method('commit')->willReturn(true);
$pdoMock->expects($this->never())->method('rollBack');
$resultado = $entrega->manejarAccion('eliminar', json_encode(['id' => 5]));
$this->assertTrue($resultado['status']);
$this->assertEquals('Entrega eliminada con éxito.', $resultado['msj']);
$this->assertEquals(5, $resultado['data_bitacora']['id_entregas']);
```

Evidencia: ver Captura 8 (ejecución de la suite completa).

9. Prueba unitaria UT-05: Cancelar entrega sin motivo es rechazada

ID	UT-05
Nombre	Cancelar entrega sin motivo es rechazada
Tipo de prueba	Unitaria (aislada con mocks)
Acción evaluada	cancelar
Archivo y método	tests/EntregaTest.php · test_cancelar_entrega_sin_motivo_es_rechazada
Descripción	Verifica que no se pueda cancelar una entrega sin indicar el motivo, y que el rechazo ocurra antes de consultar la base de datos.
Precondiciones	XAMPP apagado (Apache y MySQL detenidos) · Entrega.php corregido.
Simulación (mock)	Doble de Entrega sin constructor real. Se verifica que getConnectionNegocio nunca se ejecute.
Datos de entrada	`Acción: cancelar · JSON: {"id": 5, "motivo": ""}`
Pasos	1. Se agregó el método de prueba al archivo tests/EntregaTest.php. 2. Se detuvieron Apache y MySQL en XAMPP. 3. Se ejecutó la suite completa: vendor\bin\phpunit tests\EntregaTest.php --testdox 4. Se registró la salida de la terminal.
Resultado esperado	status false con el mensaje «El motivo de cancelación es requerido.» y sin ninguna conexión a la base de datos.
Resultado obtenido	✔️ Cancelar entrega sin motivo es rechazada (ejecutada dentro de la suite completa). El modelo rechazó la cancelación con el mensaje esperado y no intentó conectarse a la base de datos.
Estado	**Aprobado**
Fecha de ejecución	17-10-2026
Ejecutado por	Elías Armas

Código de la prueba:

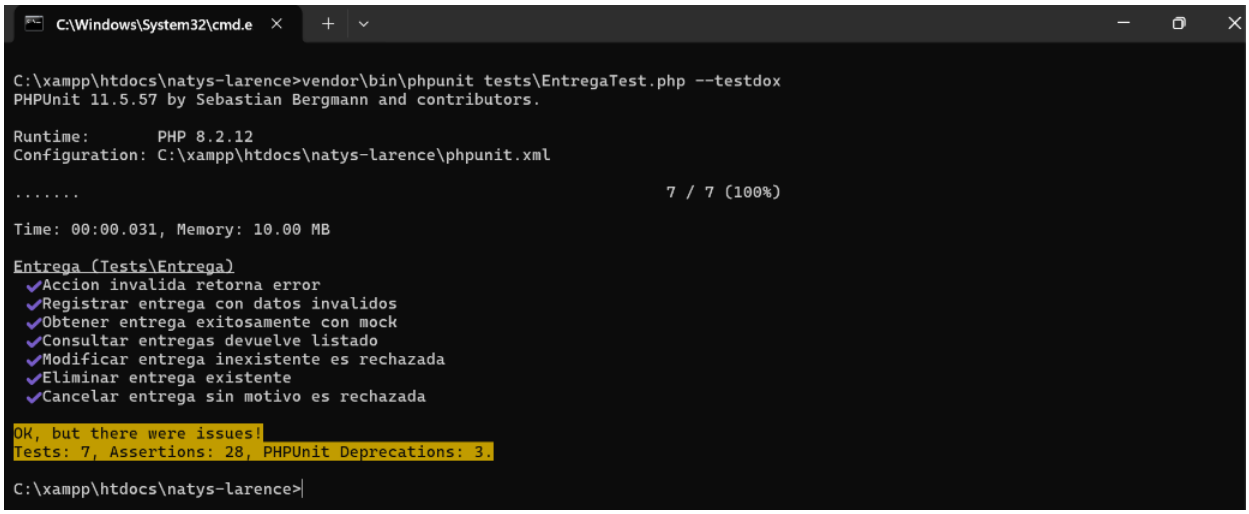
```
$entrega->expects($this->never())->method('getConnectionNegocio');
$resultado = $entrega->manejarAccion('cancelar', json_encode(['id' => 5, 'motivo' => '']));
$this->assertFalse($resultado['status']);
$this->assertEquals('El motivo de cancelación es requerido.', $resultado['msj']);
```

Evidencia: ver Captura 8 (ejecución de la suite completa).

10. Ejecución completa de la suite

Comando	vendor\bin\phpunit tests\EntregaTest.php --testdox
Precondiciones	XAMPP apagado (Apache y MySQL detenidos).
Resultado esperado	Las 7 pruebas del modelo Entrega aprobadas sin conexión a la base de datos.
Resultado obtenido	7 de 7 pruebas aprobadas, 28 aserciones, sin fallos (0,031 s; 10 MB). PHPUnit mostró 3 avisos «PHPUnit Deprecations» por el uso de la anotación /** @test */ en comentarios, un estilo que PHPUnit 11 marca como obsoleto. No afectan los resultados.
Estado	**Aprobado**
Fecha de ejecución	17-10-2026
Ejecutado por	Elías Armas

Captura 8 – Ejecución de la suite completa



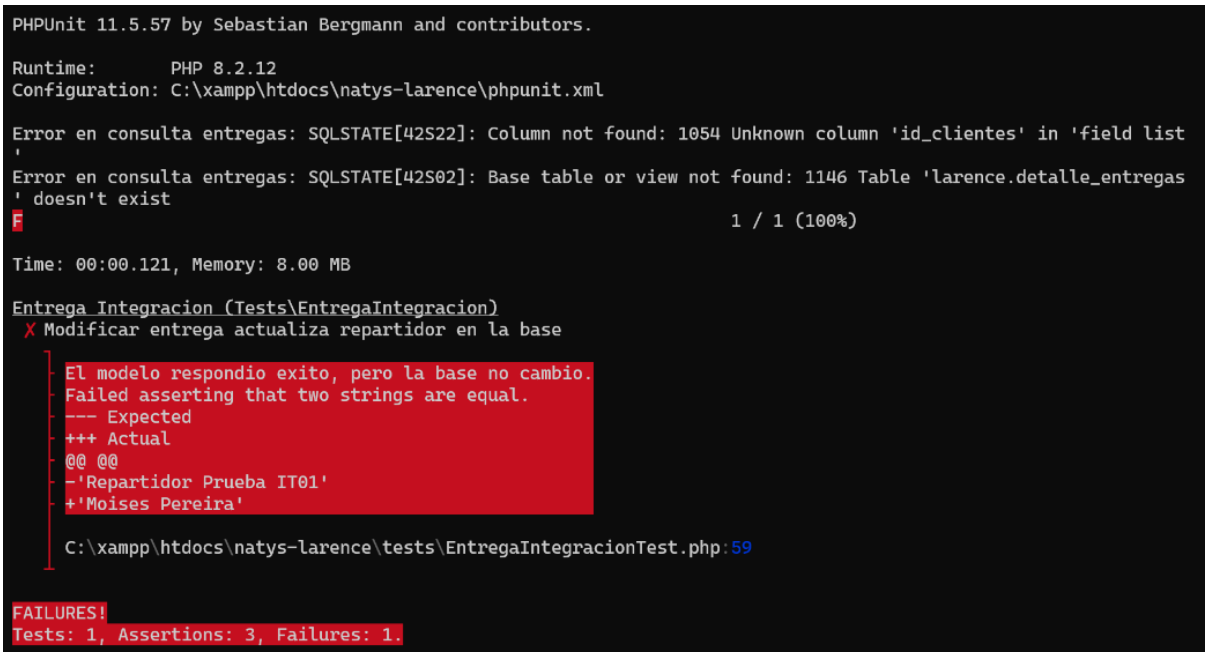
7 pruebas aprobadas con XAMPP apagado

11. Prueba de integración IT-01

A diferencia de las pruebas unitarias, la prueba de integración usa el modelo Entrega real conectado a la base de datos MySQL, para comprobar que el código y la base trabajan correctamente juntos.

ID	IT-01
Nombre	Modificar entrega actualiza el repartidor en la base de datos
Tipo de prueba	Integración (modelo Entrega + MySQL)
Herramienta	PHPUnit 11.5.57
Archivo y método	tests/EntregaIntegracionTest.php · test_modificar_entrega_actualiza_repartidor_en_la_base
Descripción	Toma una entrega real activa y no cancelada, le cambia el repartidor mediante la acción modificar del modelo y luego consulta directamente la tabla entregas para confirmar que el cambio quedó guardado.
Precondiciones	MySQL activo en XAMPP · base larence con al menos una entrega activa con detalle.
Datos de entrada	`Acción: modificar · id, pedido, cliente, dirección, fecha y estado tomados de una entrega real · repartidor: «Repartidor Prueba IT01»`
Pasos	1. Se encendió MySQL en XAMPP. 2. Se creó el archivo tests/EntregaIntegracionTest.php. 3. Se ejecutó: vendor\bin\phpunit tests\EntregaIntegracionTest.php --testdox 4. Se registró la salida de la terminal.
Resultado esperado	El modelo devuelve status true y la columna repartidor de la entrega queda con el valor «Repartidor Prueba IT01».
Resultado obtenido	El modelo respondió éxito, pero la base no cambió: el repartidor siguió siendo «Moises Pereira». PHPUnit reportó «Failed asserting that two strings are equal» (EntregaIntegracionTest.php, línea 59). MySQL registró dos errores: 1054 Unknown column 'id_clientes' in 'field list' y 1146 Table 'larence.detalle_entregas' doesn't exist. Resumen: 1 prueba, 3 aserciones, 1 fallo.
Estado	**Fallido (defecto encontrado)**
Fecha de ejecución	17-10-2026
Ejecutado por	Elías Armas

Captura 9 – Resultado obtenido de IT-01 antes de la corrección



Errores 1054 y 1146 y fallo de la aserción

11.1 Hallazgo BUG-02

ID	BUG-02
Caso relacionado	IT-01 – Modificar entrega actualiza el repartidor
Descripción	Al modificar una entrega, el modelo intentaba actualizar en la tabla entregas columnas que no existen en ella (id_clientes, entre otras) y guardar el detalle en la tabla detalle_entregas, que no existe (la correcta es detalles_entregas). Los errores se capturaban sin propagarse, por lo que el modelo respondía «Entrega actualizada con éxito» sin haber guardado nada.
Severidad	Alta
Detectado por	Prueba de integración (no detectable con mocks, ya que la base de datos era simulada)
Estado	**Corregido**

11.2 Corrección aplicada

Archivo	Función	Cambio
app/models/Entrega.php	Actualizar_Cabecera_Entrega	Solo actualiza columnas existentes en entregas: fecha programada, repartidor, observaciones y estado. En el catch se reemplazó el return por throw $e;.
app/models/Entrega.php	Guardar_Detalle_Entrega_Individual	Usa la tabla real detalles_entregas, conserva los valores guardados cuando un dato llega vacío, no modifica entregas con varios pedidos (se usa modificar_lote) y propaga los errores con throw $e;.

Consulta corregida en Actualizar_Cabecera_Entrega:

```
UPDATE entregas
SET fecha_entrega_programada = :fecha_programada,
    repartidor = :repartidor,
    observaciones = :observaciones,
    id_estado_entrega = :estado_id
WHERE id_entregas = :id AND status = 1
```

11.3 Re-ejecución de IT-01

ID	IT-01 (R)
Comando	vendor\bin\phpunit tests\EntregaIntegracionTest.php --testdox
Resultado esperado	El repartidor queda guardado en la base de datos y no aparecen errores de MySQL.
Resultado obtenido	✔️ Modificar entrega actualiza repartidor en la base. OK: 1 prueba, 3 aserciones (0,080 s; 8 MB). Ya no aparecen los errores 1054 y 1146.
Estado	**Aprobado**
Fecha de ejecución	17-10-2026
Ejecutado por	Elías Armas

Captura 10 – Re-ejecución de IT-01 después de la corrección

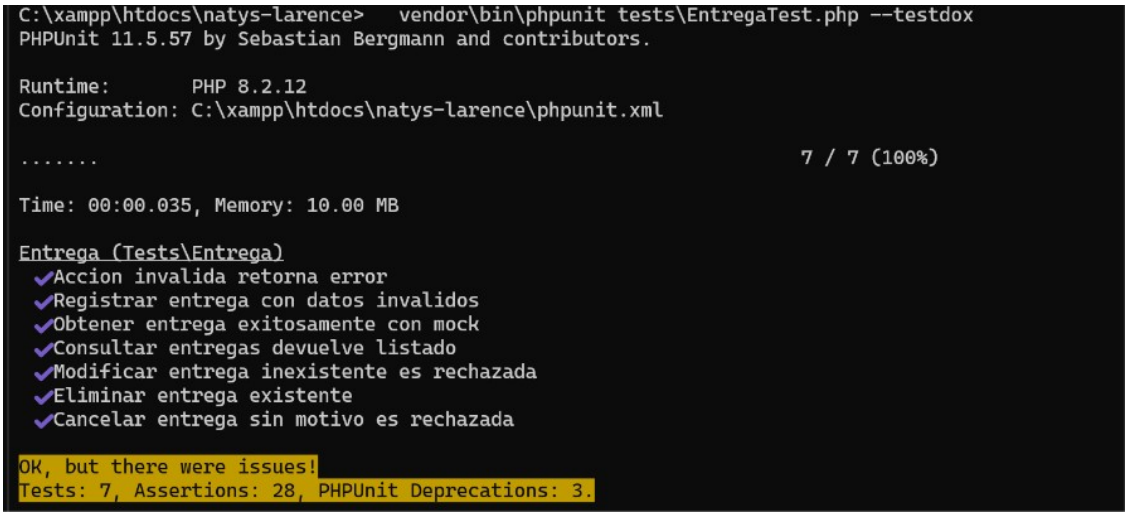


Prueba de integración aprobada

11.4 Regresión de las pruebas unitarias

Comando	vendor\bin\phpunit tests\EntregaTest.php --testdox
Resultado esperado	Las 7 pruebas unitarias siguen aprobadas después de las correcciones.
Resultado obtenido	7 de 7 pruebas aprobadas, 28 aserciones, sin fallos (0,035 s; 10 MB).
Estado	**Aprobado**

Captura 11 – Regresión de la suite unitaria



7 pruebas aprobadas después de las correcciones

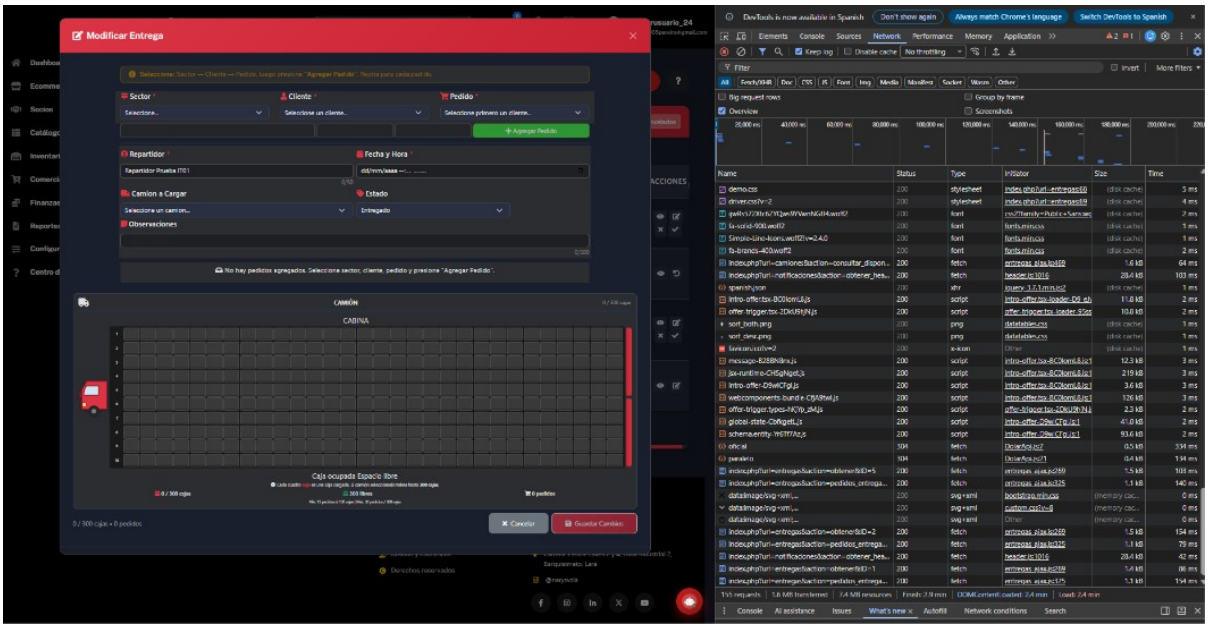
12. Hallazgo BUG-03: observaciones borradas al modificar

Al revisar en la interfaz la entrega usada en IT-01, el formulario de edición mostraba vacíos el sector, las cajas y las observaciones. Natys guarda el sector y las cajas dentro del campo observaciones con el formato [SECTOR:x][CAJAS:n].

ID	BUG-03
Detectado en	Revisión manual de la interfaz después de la re-ejecución de IT-01
Descripción	La primera corrección del BUG-02 actualizaba siempre la columna observaciones. Como IT-01 no enviaba ese dato, la modificación lo dejó vacío y se perdieron el sector y las cajas de la entrega.
Causa	La consulta UPDATE asignaba directamente :observaciones, sin conservar el valor anterior cuando llegaba vacío. La prueba IT-01 solo verificaba el repartidor.
Corrección	En Actualizar_Cabecera_Entrega, la fecha, el repartidor y las observaciones usan COALESCE(NULLIF(:valor, ''), columna): si el dato llega vacío se conserva el guardado. Además, IT-01 ahora verifica que las observaciones no cambien.
Severidad	Alta (pérdida de datos)
Estado	**Corregido**

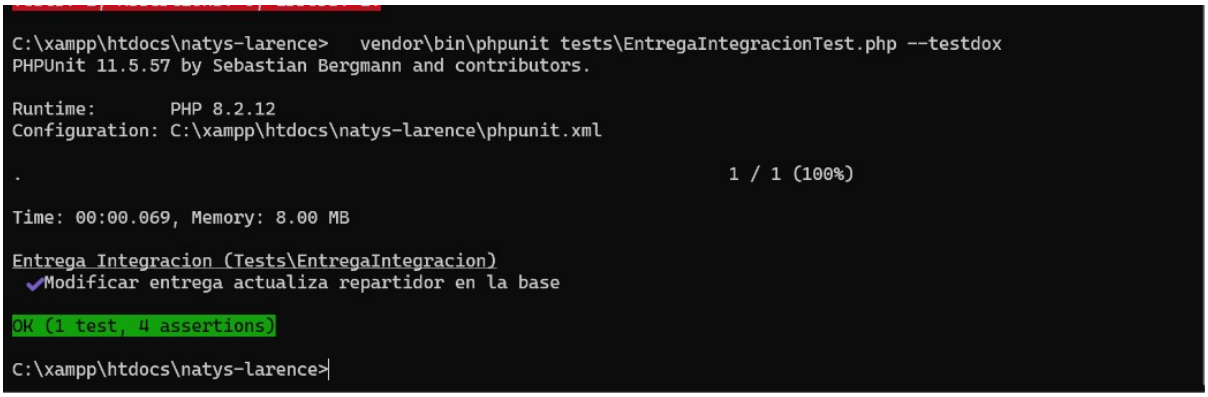
```
SET fecha_entrega_programada = COALESCE(NULLIF(:fecha_programada, ''), fecha_entrega_programada),
    repartidor               = COALESCE(NULLIF(:repartidor, ''), repartidor),
    observaciones            = COALESCE(NULLIF(:observaciones, ''), observaciones),
    id_estado_entrega        = :estado_id
```

Captura 12 – Entrega modificada por IT-01 sin sector ni cajas



Formulario de edición antes de la corrección

Captura 13 – Re-ejecución de IT-01 reforzada



OK: 1 prueba, 4 aserciones; las observaciones se conservan

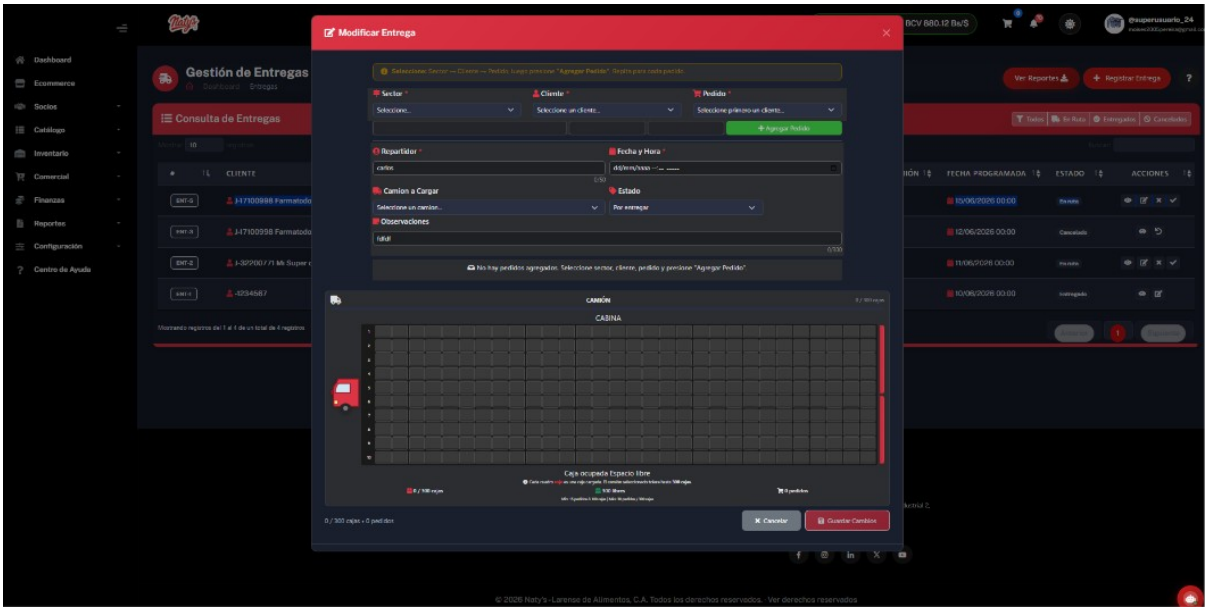
13. Hallazgo BUG-04: el formulario de edición no carga los datos

Al abrir Modificar en una entrega registrada de forma individual (por ejemplo EMT-5, Farmatodo, pedido #16), el formulario no mostraba el sector, la lista de pedidos ni la fecha, aunque el servidor sí respondía (código 200).

Dato	Causa	Archivo	Estado
Sector	La consulta de la acción obtener devolvía siempre NULL as sector.	Entrega.php	**Corregido**
Lista de pedidos	pedidos_entrega solo buscaba en entrega_pedidos; las entregas individuales guardan su pedido en detalles_entregas.	Entrega.php	**Corregido**
Fecha y hora	La fecha llega sin hora (AAAA-MM-DD) y el campo datetime-local exige AAAA-MM-DDTHH:MM. Se agregó la función formatoFechaInput; el campo aún no muestra la fecha y queda en análisis.	entregas_ajax.js	**Pendiente**

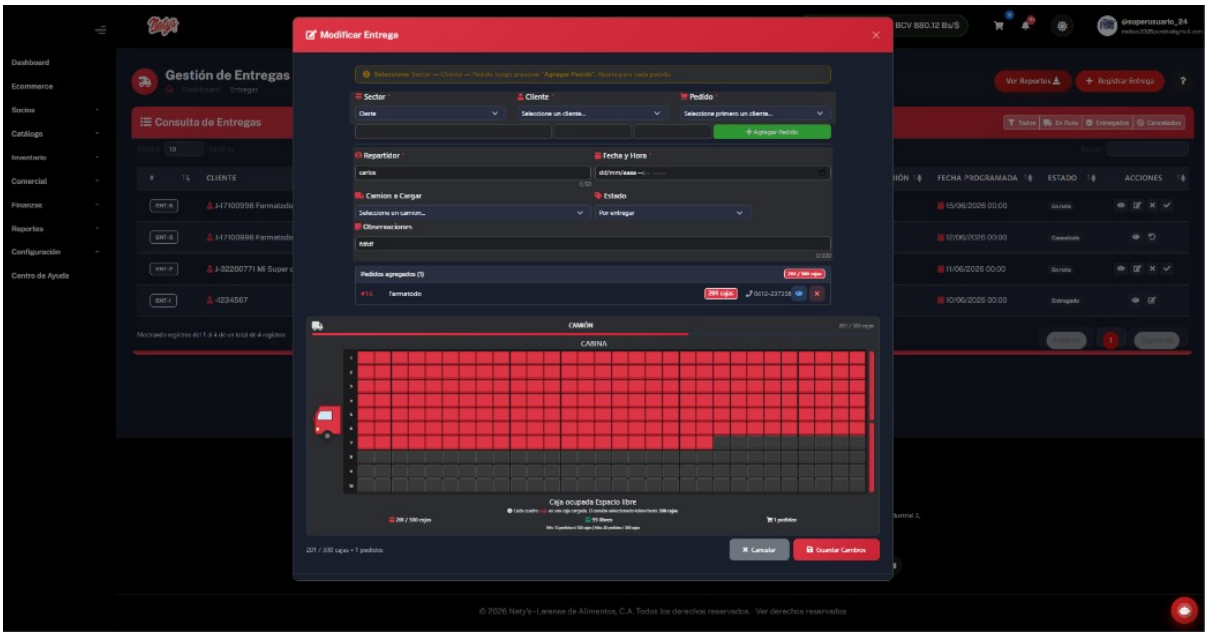
ID	BUG-04
Detectado en	Revisión manual de la interfaz (Modificar entrega)
Severidad	Media
Estado	**Corregido parcialmente: sector y pedidos corregidos; fecha pendiente**

Captura 14 – Formulario de edición antes de la corrección



Sector, pedidos y fecha vacíos

Captura 15 – Formulario de edición después de la corrección



Sector «Oeste», pedido #16 con 201 cajas y camión cargado

14. Prueba de backend (API) API-01

La prueba de backend se comunica directamente con el controlador de Entregas mediante peticiones HTTP, sin usar la interfaz. Se utilizó Postman para enviar la petición y validar la respuesta de forma automática.

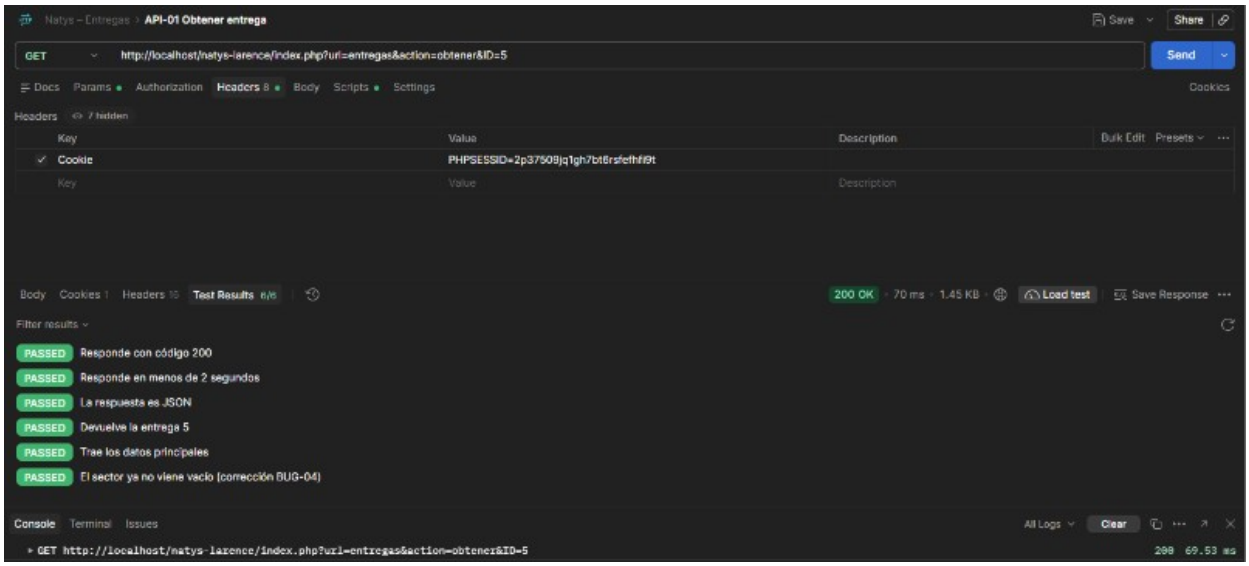
ID	API-01
Nombre	Obtener una entrega por su ID
Tipo de prueba	Backend (API)
Herramienta	Postman · colección «Natys – Entregas»
Petición	`GET http://localhost/natys-larence/index.php?url=entregas&action=obtener&ID=5`
Descripción	Verifica que el endpoint obtener del EntregaController responda con los datos de la entrega 5 en formato JSON, de forma rápida y con el sector corregido en el BUG-04.
Precondiciones	Apache y MySQL activos en XAMPP · sesión iniciada en Natys · cookie de sesión PHPSESSID copiada desde el navegador.
Datos de entrada	`Encabezado: Cookie: PHPSESSID=<sesión activa>`
Pasos	1. Se inició sesión en Natys y se copió la cookie PHPSESSID desde las herramientas del navegador (Application → Cookies). 2. En Postman se creó la colección «Natys – Entregas» y la petición GET «API-01 Obtener entrega». 3. Se agregó el encabezado Cookie con el valor PHPSESSID=<sesión>. 4. Se escribieron 6 validaciones automáticas en Scripts → Post-response. 5. Se envió la petición (Send) y se revisaron Test Results, Body y Console.
Resultado esperado	Código 200, respuesta JSON en menos de 2 segundos, id_entregas igual a 5, campos repartidor, fecha_programada y estado presentes, y sector distinto de null.
Resultado obtenido	6 de 6 validaciones aprobadas. Código 200 OK en 70 ms; respuesta JSON de 1,45 KB con la entrega 5 (Farmatodo, pedido 16, sector «Oeste»). La consola de Postman muestra una única petición GET con estado 200, sin redirecciones.
Estado	**Aprobado**
Fecha de ejecución	17-10-2026

Ejecutado por	Elías Armas

Validaciones automáticas (Scripts → Post-response):

```
pm.test("Responde con código 200", () => pm.response.to.have.status(200));
pm.test("Responde en menos de 2 segundos", () =>
pm.expect(pm.response.responseTime).to.be.below(2000));
pm.test("La respuesta es JSON", () => pm.response.to.have.jsonBody());
pm.test("Devuelve la entrega 5", () => pm.expect(pm.response.json().id_entregas).to.eql(5));
pm.test("Trae los datos principales", () => { /* repartidor, fecha_programada, estado */ });
pm.test("El sector ya no viene vacío (corrección BUG-04)", () =>
pm.expect(pm.response.json().sector).to.not.be.null);
```

Captura 16 – Resultado de API-01 en Postman

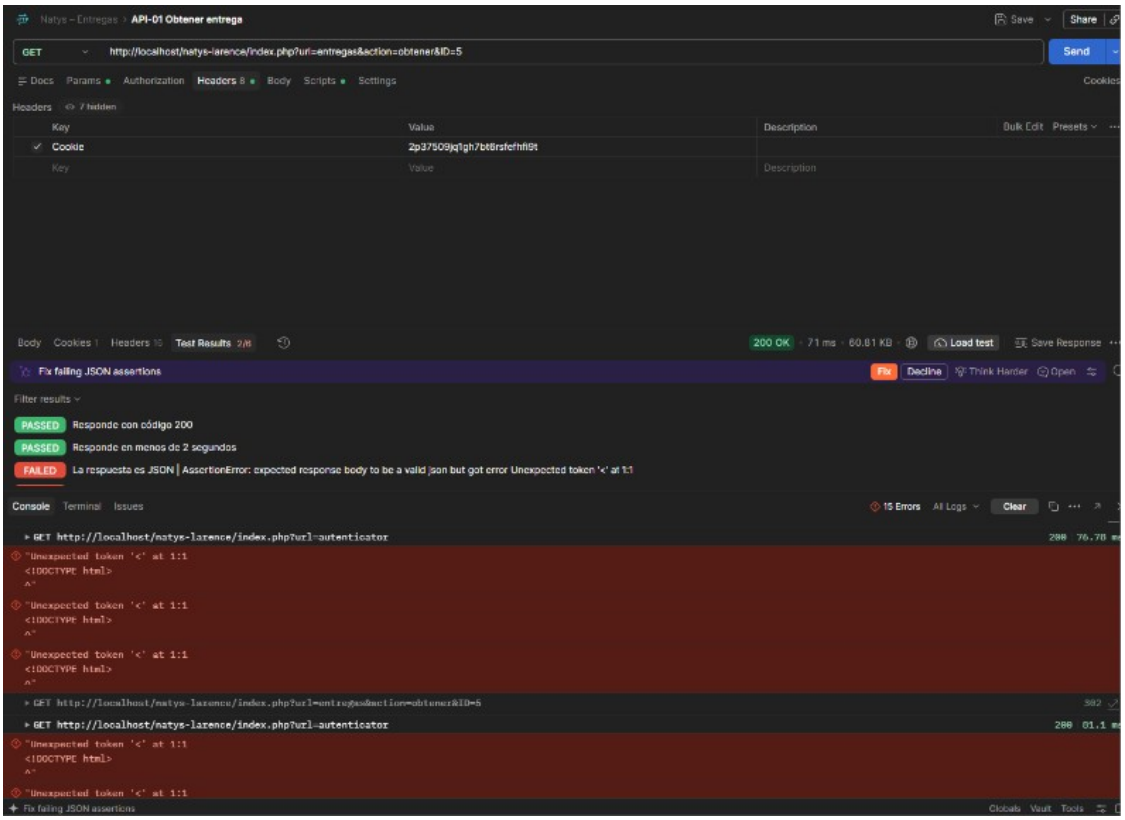


6 validaciones aprobadas, 200 OK y una sola petición en la consola

14.1 Observación: control de acceso del backend

Situación	En los primeros envíos la sesión había expirado por inactividad y, después, la cookie se envió sin el nombre PHPSESSID.
Comportamiento observado	El servidor respondió 302 y redirigió a index.php?url=autenticator (página de inicio de sesión). Postman recibió HTML en lugar de JSON y solo 2 de 6 validaciones pasaron.
Conclusión	No es un defecto: el backend protege el endpoint y no entrega datos sin una sesión válida. Es el comportamiento esperado de seguridad (RNF 03).

Captura 17 – Petición sin sesión válida



Redirección 302 al inicio de sesión y respuesta HTML

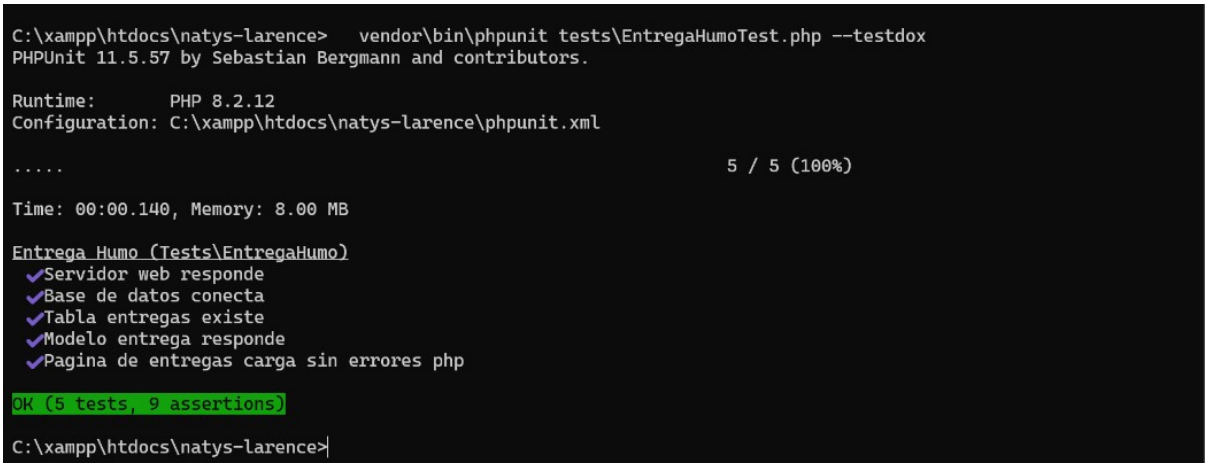
15. Prueba de humo SM-01

La prueba de humo es una verificación rápida que se ejecuta antes de las pruebas largas para confirmar que el sistema «enciende»: servidor, base de datos, modelo y página del módulo. Si falla, no tiene sentido continuar con las demás pruebas.

ID	SM-01
Nombre	El módulo de Entregas enciende correctamente
Tipo de prueba	Humo (smoke test)
Herramienta	PHPUnit 11.5.57
Archivo	tests/EntregaHumoTest.php (5 verificaciones)
Descripción	Comprueba en menos de un segundo que el servidor web responde, que la base de datos conecta, que existe la tabla entregas, que el modelo Entrega responde y que la página del módulo carga sin errores de PHP.
Precondiciones	Apache y MySQL encendidos en XAMPP.
Pasos	1. Se encendieron Apache y MySQL en XAMPP. 2. Se creó el archivo tests/EntregaHumoTest.php. 3. Se ejecutó: vendor\bin\phpunit tests\EntregaHumoTest.php --testdox 4. Se registró la salida de la terminal.
Resultado esperado	Las 5 verificaciones aprobadas en pocos milisegundos.
Resultado obtenido	OK: 5 pruebas y 9 aserciones aprobadas en 0,140 s (8 MB).
Estado	**Aprobado**
Fecha de ejecución	17-10-2026
Ejecutado por	Elías Armas

#	Verificación	Si fallara, indicaría…	Resultado
1	El servidor web responde (código 200)	Apache apagado	**Aprobado**
2	La base de datos conecta (SELECT 1)	MySQL apagado o mal configurado	**Aprobado**
3	Existe la tabla entregas	Base de datos incompleta	**Aprobado**
4	El modelo Entrega responde a consultar	Error grave en el código del modelo	**Aprobado**
5	La página de Entregas carga sin errores de PHP	Fatal error, Parse error o excepción no capturada	**Aprobado**

Captura 18 – Ejecución de la prueba de humo



5 verificaciones aprobadas, 9 aserciones

16. Prueba de interfaz y sistema UI-01

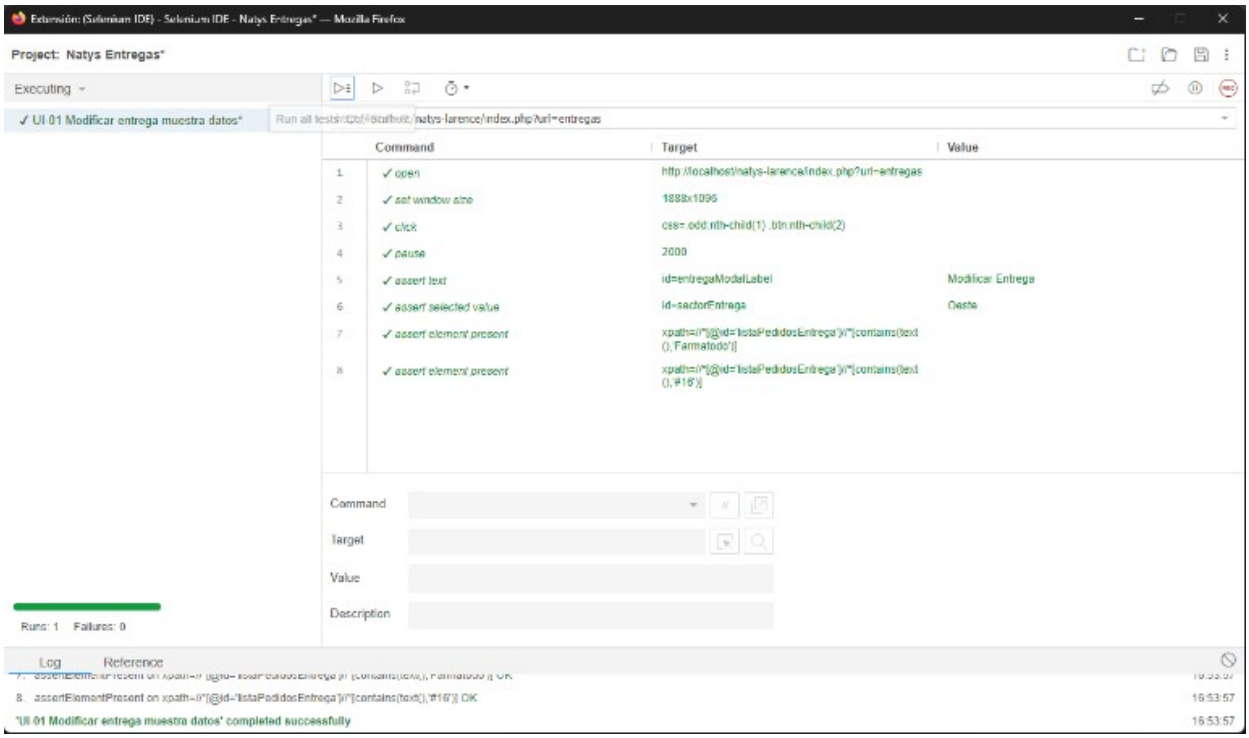
La prueba de interfaz se ejecuta desde el navegador, como lo haría un usuario real: un robot abre la pantalla, presiona los botones y verifica lo que aparece. Al recorrer el sistema completo sin mirar el código, también funciona como prueba de sistema (caja negra) del requisito RF 14.

ID	UI-01
Nombre	El formulario Modificar muestra los datos de la entrega
Tipo de prueba	Interfaz y sistema / funcional (caja negra)
Herramienta	Selenium IDE (extensión de Firefox) · proyecto Natys_Entregas.side
Descripción	Abre el módulo Entregas, presiona Modificar en la entrega EMT-5 y verifica que el formulario cargue el título, el sector y el pedido asociado. Comprueba además, desde la interfaz, la corrección del BUG-04.
Precondiciones	Apache y MySQL activos · sesión iniciada en Natys en Firefox · existe la entrega EMT-5 (Farmatodo, pedido #16).
Pasos	1. Se instaló Selenium IDE en Firefox y se creó el proyecto «Natys Entregas». 2. Se grabó la apertura del módulo Entregas y el clic en Modificar de la entrega EMT-5. 3. Se eliminaron los clics sobrantes de la grabación (incluido «Guardar Cambios»). 4. Se agregaron una espera de 2 segundos y cuatro verificaciones (assert). 5. Se ejecutó la prueba con Run current test y se revisó el registro (Log).
Resultado esperado	El robot abre el formulario «Modificar Entrega» con el sector «Oeste» y el pedido #16 de Farmatodo en la lista de pedidos agregados.
Resultado obtenido	Los 8 pasos se ejecutaron en verde. El Log indica «'UI 01 Modificar entrega muestra datos' completed successfully». Ejecuciones: 1; fallos: 0.

Estado	**Aprobado**
Fecha de ejecución	17-10-2026
Ejecutado por	Elías Armas

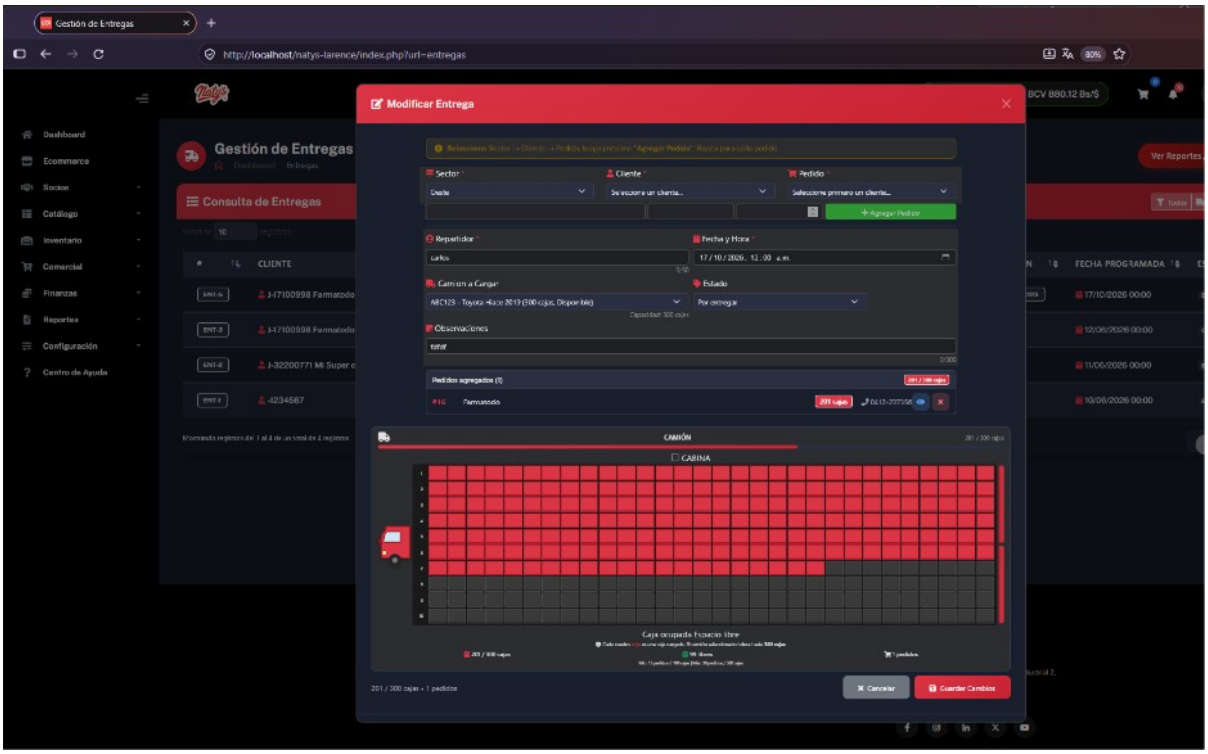
#	Comando	Objetivo	Qué hace
1	open	index.php?url=entregas	**Abre el módulo Entregas**
2	set window size	1888x1096	**Fija el tamaño de la ventana**
3	click	css=.odd:nth-child(1) .btn:nth-child(2)	**Presiona Modificar en EMT-5**
4	pause	2000	**Espera la carga de datos**
5	assert text	id=entregaModalLabel = «Modificar Entrega»	**Verifica el título**
6	assert selected value	id=sectorEntrega = «Oeste»	**Verifica el sector**
7	assert element present	listaPedidosEntrega contiene «Farmatodo»	**Verifica el cliente**
8	assert element present	listaPedidosEntrega contiene «#16»	**Verifica el pedido**

Captura 19 – Selenium IDE con la prueba UI-01 aprobada



8 pasos en verde y «completed successfully» en el Log

Captura 20 – Formulario abierto por el robot durante la ejecución



Sector «Oeste», pedido #16 Farmatodo y camión cargado

17. Pruebas de seguridad

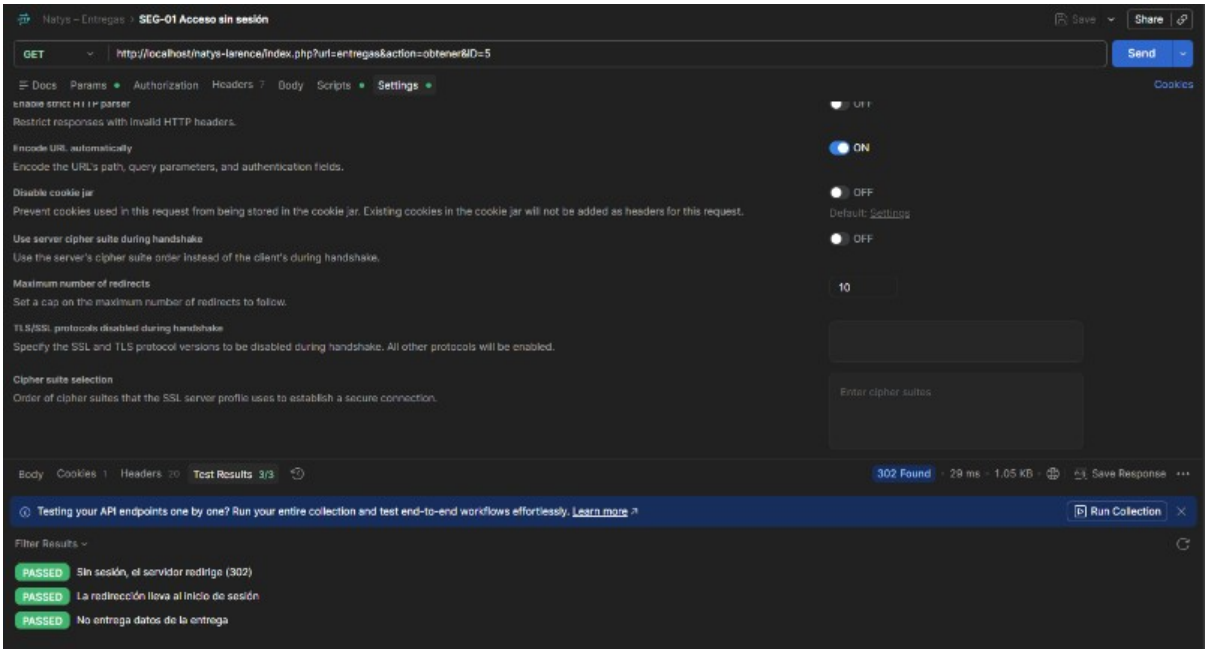
Las pruebas de seguridad intentan usar el sistema de forma indebida para comprobar que se defiende. Se ejecutaron en Postman sobre la copia local del sistema y cubren el RNF 03 (seguridad) y el RNF 04 (validación de datos).

17.1 SEG-01: acceso sin sesión

ID	SEG-01
Nombre	Un usuario sin sesión no puede obtener datos de entregas
Tipo de prueba	Seguridad (control de acceso)
Herramienta	Postman
Petición	`GET http://localhost/natys-larence/index.php?url=entregas&action=obtener&ID=5`
Precondiciones	Apache y MySQL activos · petición sin cookie de sesión · cookies de Postman borradas · opción «Automatically follow redirects» desactivada.
Pasos	1. Se duplicó la petición API-01 y se renombró como SEG-01. 2. Se desmarcó el encabezado Cookie y se borraron las cookies de localhost en Postman. 3. Se desactivó el seguimiento automático de redirecciones. 4. Se escribieron 3 validaciones y se envió la petición.
Resultado esperado	El servidor responde 302 y redirige a index.php?url=autenticator (inicio de sesión), sin incluir datos de la entrega.
Resultado obtenido	302 Found en 29 ms (1,05 KB). La cabecera Location apunta a la página de inicio de sesión y la respuesta no contiene datos de la entrega. 3 de 3 validaciones aprobadas.
Estado	**Aprobado**
Fecha de ejecución	17-10-2026
Ejecutado por	Elías Armas

```
pm.test("Sin sesión, el servidor redirige (302)", () => pm.response.to.have.status(302));
pm.test("La redirección lleva al inicio de sesión", () =>
    pm.expect(pm.response.headers.get("Location")).to.include("autenticator"));
pm.test("No entrega datos de la entrega", () =>
    pm.expect(pm.response.text()).to.not.include("id_entregas"));
```

Captura 21 – SEG-01 en Postman



302 Found y 3 validaciones aprobadas

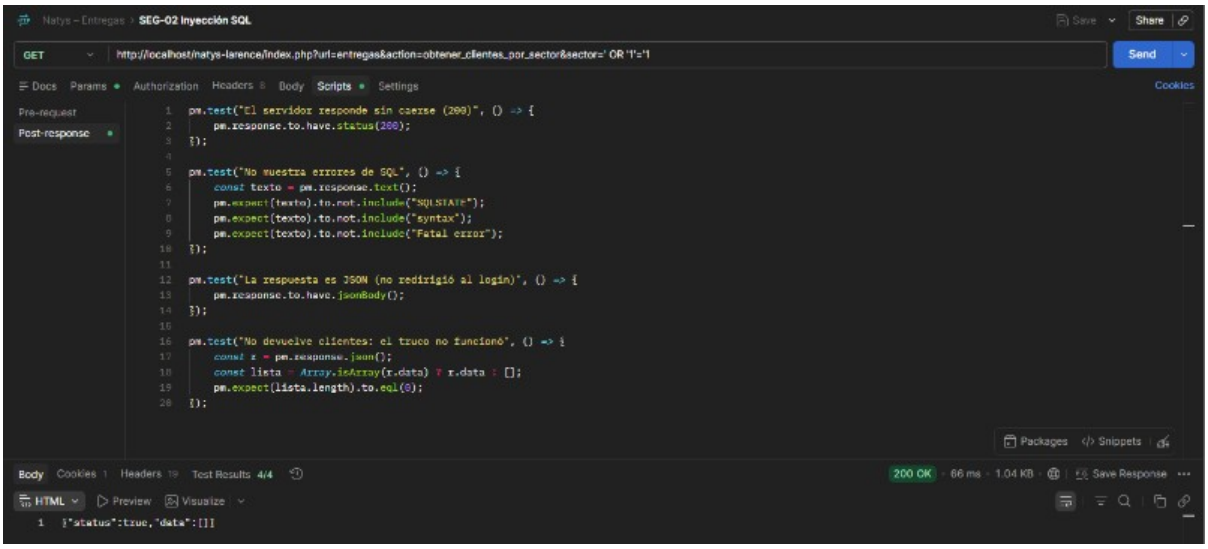
17.2 SEG-02: inyección SQL

ID	SEG-02
Nombre	El filtro de clientes por sector resiste la inyección SQL
Tipo de prueba	Seguridad (inyección SQL)
Herramienta	Postman
Petición	`GET index.php?url=entregas&action=obtener_clientes_por_sector&sector=' OR '1'='1`
Descripción	Envía en el parámetro sector una cadena de inyección SQL clásica. Un sistema vulnerable devolvería todos los clientes o un error de base de datos; uno seguro la trata como texto y no encuentra coincidencias.
Precondiciones	Apache y MySQL activos · sesión iniciada (cookie PHPSESSID válida en el encabezado).
Datos de entrada	`sector = ' OR '1'='1`
Pasos	1. Se duplicó la petición API-01 y se renombró como SEG-02. 2. Se cambió la URL a la acción obtener_clientes_por_sector y se agregó el parámetro sector con la cadena de inyección. 3. Se actualizó la cookie de sesión. 4. Se escribieron 4 validaciones y se envió la petición.
Resultado esperado	Código 200, respuesta JSON, sin mensajes de error SQL y sin clientes en la lista.
Resultado obtenido	200 OK en 66 ms. Respuesta: {"status":true,"data":[]}: la lista de clientes llegó vacía y no aparecieron errores SQL. 4 de 4 validaciones aprobadas. La consulta usa parámetros preparados (PDO), por lo que la cadena se trató como un valor de texto y no como parte de la instrucción SQL.

Estado	**Aprobado**
Fecha de ejecución	17-10-2026
Ejecutado por	Elías Armas

```
pm.test("El servidor responde sin caerse (200)", () => pm.response.to.have.status(200));
pm.test("No muestra errores de SQL", () => { /* SQLSTATE, syntax, Fatal error */ });
pm.test("La respuesta es JSON (no redirigió al login)", () => pm.response.to.have.jsonBody());
pm.test("No devuelve clientes: el truco no funcionó", () => {
    const r = pm.response.json();
    pm.expect((Array.isArray(r.data) ? r.data : []).length).to.eql(0);
});
```

Captura 22 – SEG-02 en Postman



4 validaciones aprobadas y lista de clientes vacía

18. Prueba de rendimiento RD-01

La prueba de rendimiento mide cómo responde el sistema cuando muchos usuarios lo usan al mismo tiempo. Se utilizó Apache JMeter para simular usuarios virtuales que consultan una entrega de forma simultánea (RNF 05).

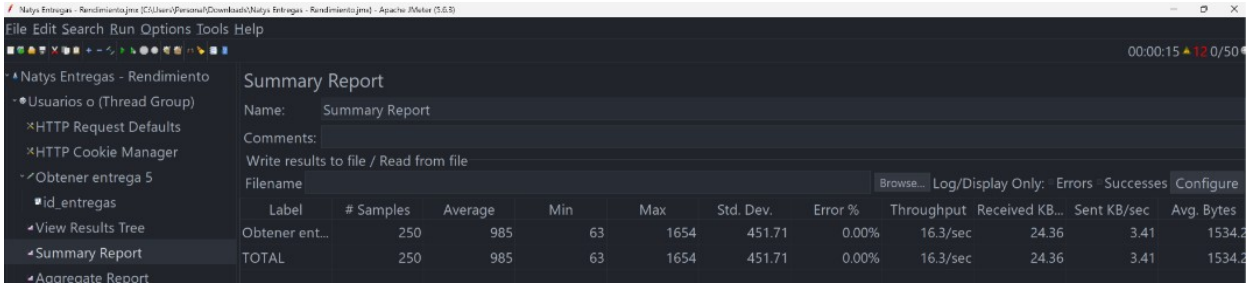
ID	RD-01
Nombre	50 usuarios consultan una entrega al mismo tiempo
Tipo de prueba	Rendimiento / carga (no funcional)
Herramienta	Apache JMeter 5.6.3 · plan Natys_Entregas_Rendimiento.jmx
Petición	`GET http://localhost/natys-larence/index.php?url=entregas&action=obtener&ID=5`
Configuración	Thread Group: 50 usuarios virtuales · ramp-up de 10 segundos · 5 repeticiones por usuario (250 peticiones en total).
Precondiciones	Apache y MySQL activos · cookie de sesión PHPSESSID válida configurada en JMeter.
Pasos	1. Se creó el plan de prueba «Natys Entregas - Rendimiento». 2. Se agregaron el Thread Group, HTTP Request Defaults (localhost:80) y la cookie de sesión. 3. Se agregó la petición HTTP «Obtener entrega 5» con una Response Assertion que exige el texto id_entregas en cada respuesta. 4. Se agregaron los listeners View Results Tree, Summary Report y Aggregate Report. 5. Se validó primero con 1 usuario y luego se ejecutó con 50 usuarios.

Resultado esperado	250 peticiones procesadas, tiempo promedio menor a 2.000 ms y 0 % de errores.
Resultado obtenido	250 peticiones, promedio de 985 ms (mínimo 63 ms, máximo 1.654 ms), desviación estándar de 451,71 ms, 0,00 % de errores y un rendimiento de 16,3 peticiones por segundo. Todas las respuestas contenían los datos de la entrega.
Estado	**Aprobado**
Fecha de ejecución	17-10-2026
Ejecutado por	Elías Armas

Métrica	Meta	Obtenido	Resultado
Peticiones procesadas	250	250	**Cumple**
Tiempo promedio	< 2.000 ms	985 ms	**Cumple**
Tiempo máximo	—	1.654 ms	**Bajo 2 s**
Errores	0 %	0,00 %	**Cumple**
Rendimiento (throughput)	—	16,3 pet./s	**Referencia**

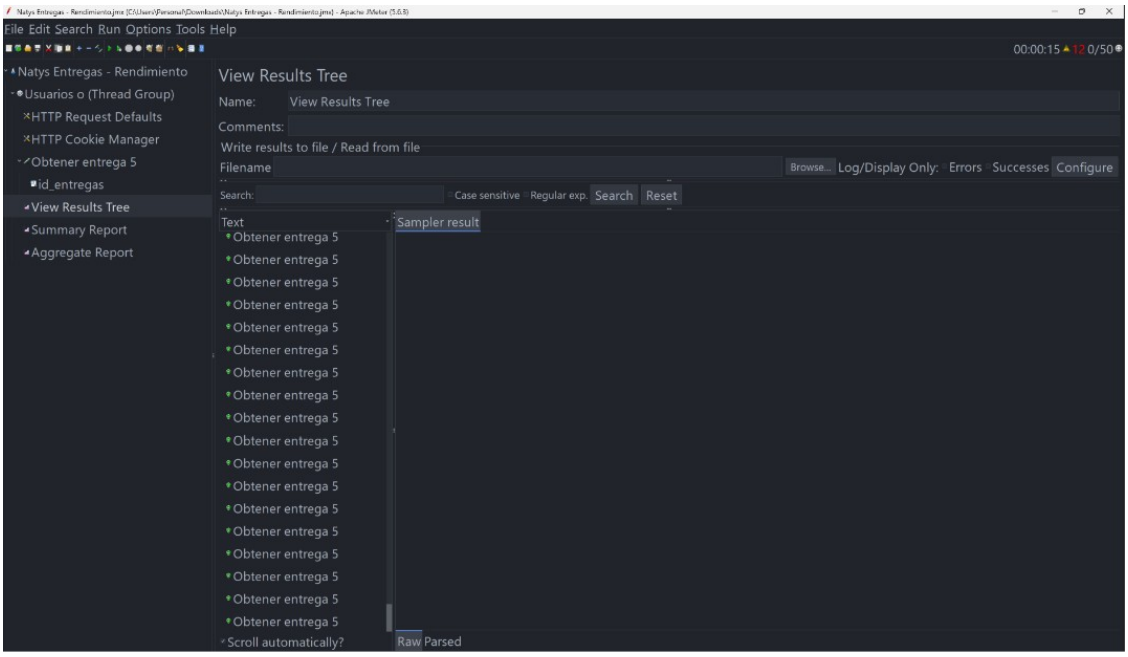
Limitación	Los 50 usuarios virtuales comparten la misma cookie de sesión. PHP bloquea la sesión mientras atiende cada petición, por lo que las peticiones se procesan en fila; los tiempos obtenidos son, por lo tanto, más altos que con 50 usuarios reales con sesiones distintas. Aun así, el sistema cumplió la meta.

Captura 23 – Summary Report de JMeter



250 peticiones, promedio de 985 ms y 0,00 % de errores

Captura 24 – View Results Tree



Todas las peticiones «Obtener entrega 5» en verde

19. Prueba de aceptación AC-01

La prueba de aceptación comprueba que el módulo cumple lo que el cliente pidió en el SRS. Se escribió con Codeception, que describe la prueba como una historia de usuario («yo entro a…, yo veo…»), y se ejecutó con PhpBrowser, un navegador sin interfaz que recorre el sistema real.

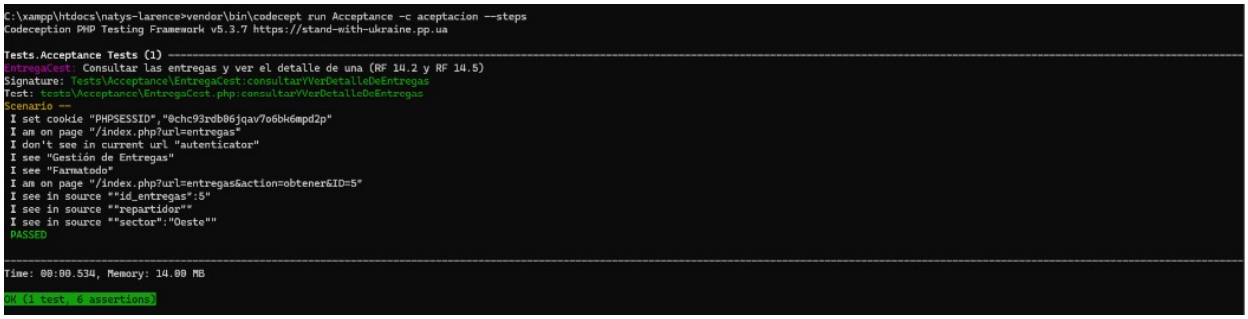
ID	AC-01
Nombre	El usuario consulta las entregas y ve el detalle de una
Tipo de prueba	Aceptación
Herramienta	Codeception 5.3.7 con el módulo PhpBrowser
Requisitos cubiertos	RF 14.2 (Consultar entregas) y RF 14.5 (Ver detalle de una entrega)
Archivo y método	tests/aceptacion/tests/Acceptance/EntregaCest.php · consultarYVerDetalleDeEntregas
Precondiciones	Apache y MySQL activos · sesión iniciada en Natys (cookie PHPSESSID) · existe la entrega EMT-5 (Farmatodo, sector Oeste).
Pasos	1. Se descargó Composer en el proyecto y se activó la extensión zip de PHP en php.ini. 2. Se instaló Codeception con: php composer.phar require --dev codeception/codeception codeception/module-phpbrowser codeception/module-asserts 3. Se creó la estructura de pruebas con: vendor\bin\codecept bootstrap aceptacion (luego la carpeta se movió a tests\aceptacion) 4. Se configuró la URL http://localhost/natys-larence en Acceptance.suite.yml y se creó EntregaCest.php. 5. Se ejecutó: vendor\bin\codecept run Acceptance -c tests\aceptacion --steps
Resultado esperado	El usuario con sesión ve el listado con el título «Gestión de Entregas» y la entrega de Farmatodo, y al pedir el detalle de la entrega 5 obtiene su id, el repartidor y el sector «Oeste».
Resultado obtenido	Escenario PASSED: los 9 pasos de la historia se cumplieron. OK (1 prueba, 6 aserciones) en 0,534 s (14 MB).
Estado	**Aprobado**
Fecha de ejecución	17-10-2026
Ejecutado por	Elías Armas

Requisito	Criterio de aceptación	Verificación en Codeception	Resultado
RF 14.2	El usuario con sesión ve el listado de entregas	amOnPage + dontSeeInCurrentUrl('autenticator') + see('Gestión de Entregas') + see('Farmatodo')	**Cumple**
RF 14.5	El usuario obtiene el detalle completo de una entrega	seeInSource: id_entregas 5, repartidor y sector «Oeste»	**Cumple**

Escenario escrito en Codeception:

```
$I->wantTo('consultar las entregas y ver el detalle de una (RF 14.2 y RF 14.5)');
$I->setCookie('PHPSESSID', self::SESION);
$I->amOnPage('/index.php?url=entregas');
$I->dontSeeInCurrentUrl('autenticator');
$I->see('Gestión de Entregas');
$I->see('Farmatodo');
$I->amOnPage('/index.php?url=entregas&action=obtener&ID=5');
$I->seeInSource('"id_entregas":5');
$I->seeInSource('"repartidor"');
$I->seeInSource('"sector":"Oeste"');
```

Captura 25 – Ejecución de AC-01 con Codeception



Historia de pasos y resultado OK (1 prueba, 6 aserciones)

19.1 Acta de aceptación

Quien firma confirma que el Módulo de Entregas cumple los requisitos RF 14.2 y RF 14.5 según los resultados de esta prueba.

Nombre	
Rol	
Firma	
Fecha	

20. Regresión final

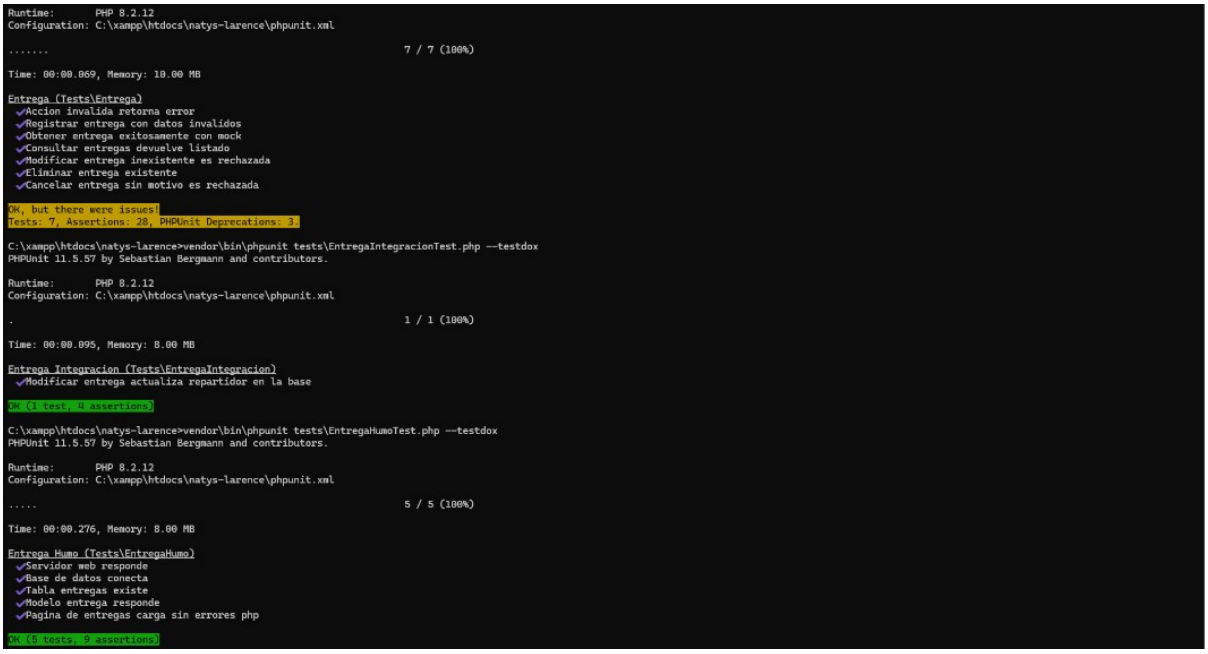
La prueba de regresión vuelve a ejecutar todas las pruebas automatizadas después de aplicar las correcciones de los BUG-01 al BUG-04, para confirmar que ningún cambio dañó lo que ya funcionaba. No se escribieron pruebas nuevas: se repitieron las existentes con sus mismas herramientas. La prueba de rendimiento no se repite, porque la regresión verifica el funcionamiento y no la velocidad.

ID	RG-01
Nombre	Regresión final del módulo de Entregas
Tipo de prueba	Regresión
Herramientas	PHPUnit, Codeception, Selenium IDE y Postman
Precondiciones	Apache y MySQL activos · Entrega.php con todas las correcciones · sesión iniciada para las pruebas que la requieren.

Pasos	1. PHPUnit: vendor\bin\phpunit tests\EntregaTest.php, tests\EntregaIntegracionTest.php y tests\EntregaHumoTest.php con --testdox. 2. Codeception: vendor\bin\codecept run Acceptance -c tests\aceptacion --steps (la carpeta de aceptación se reubicó dentro de tests). 3. Selenium IDE: se abrió Natys_Entregas.side en Firefox y se ejecutó UI-01. 4. Postman: se enviaron API-01 y SEG-02 con la cookie de sesión, y SEG-01 sin cookie.
Resultado esperado	Todas las pruebas automatizadas aprobadas, igual que en su ejecución individual.
Resultado obtenido	Todas las pruebas aprobadas. Ninguna corrección afectó funcionalidades existentes.
Estado	**Aprobado**
Fecha de ejecución	17-10-2026
Ejecutado por	Elías Armas

Herramienta	Pruebas re-ejecutadas	Resultado	Estado
PHPUnit	Unitarias (EntregaTest)	7/7 · 28 aserciones	**Aprobado**
PHPUnit	Integración (EntregaIntegracionTest)	1/1 · 4 aserciones	**Aprobado**
PHPUnit	Humo (EntregaHumoTest)	5/5 · 9 aserciones	**Aprobado**
Codeception	Aceptación AC-01	1/1 · 6 aserciones	**Aprobado**
Selenium IDE	Interfaz y sistema UI-01	8/8 pasos · 0 fallos	**Aprobado**
Postman	API-01, SEG-01 y SEG-02	13/13 validaciones	**Aprobado**

Captura 26 – Regresión con PHPUnit



Unitarias 7/7, integración 1/1 y humo 5/5

Captura 27 – Regresión con Codeception

```
C:\xampp\htdocs\natys-larence>notepad tests\aceptacion\tests\Acceptance\EntregaCest.php

C:\xampp\htdocs\natys-larence>findstr SESION tests\aceptacion\tests\Acceptance\EntregaCest.php
    private const SESION = 'b7tlu926sd7ho8lpd89bj6l1e3';
        $I->setCookie('PHPSESSID', self::SESION);

C:\xampp\htdocs\natys-larence>vendor\bin\codecept run Acceptance -c tests\aceptacion --steps
Codeception PHP Testing Framework v5.3.7 https://stand-with-ukraine.pp.ua

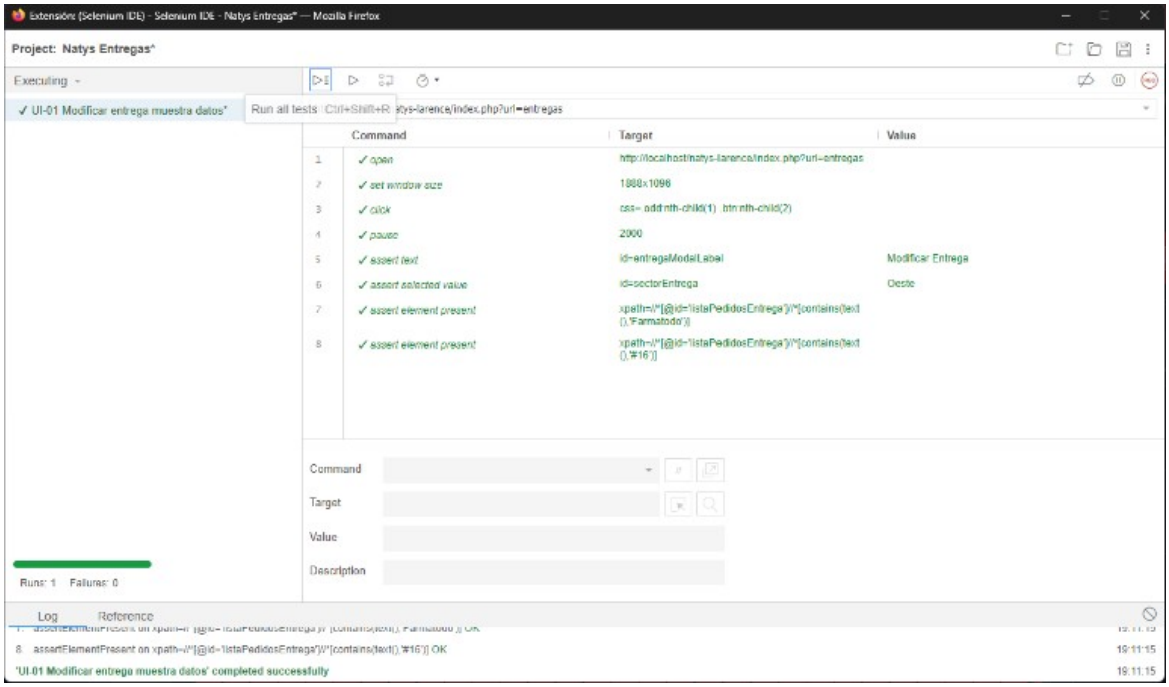
Tests.Acceptance Tests (1) ----------------------------------------------------------------------------------------------------------------------------------
EntregaCest: Consultar las entregas y ver el detalle de una (RF 14.2 y RF 14.5)
Signature: Tests\Acceptance\EntregaCest:consultarYVerDetalleDeEntregas
Test: tests\Acceptance\EntregaCest.php:consultarYVerDetalleDeEntregas
Scenario --
 I set cookie "PHPSESSID","b7tlu926sd7ho8lpd89bj6l1e3"
 I am on page "/index.php?url=entregas"
 I don't see in current url "autenticator"
 I see "Gestión de Entregas"
 I see "Farmatodo"
 I am on page "/index.php?url=entregas&action=obtener&ID=5"
 I see in source ""id_entregas":5"
 I see in source ""repartidor""
 I see in source ""sector":"Oeste""
 PASSED
----------------------------------------------------------------------------------------------------------------------------------------------------------
Time: 00:00.542, Memory: 14.00 MB

OK (1 test, 6 assertions)

C:\xampp\htdocs\natys-larence>
```

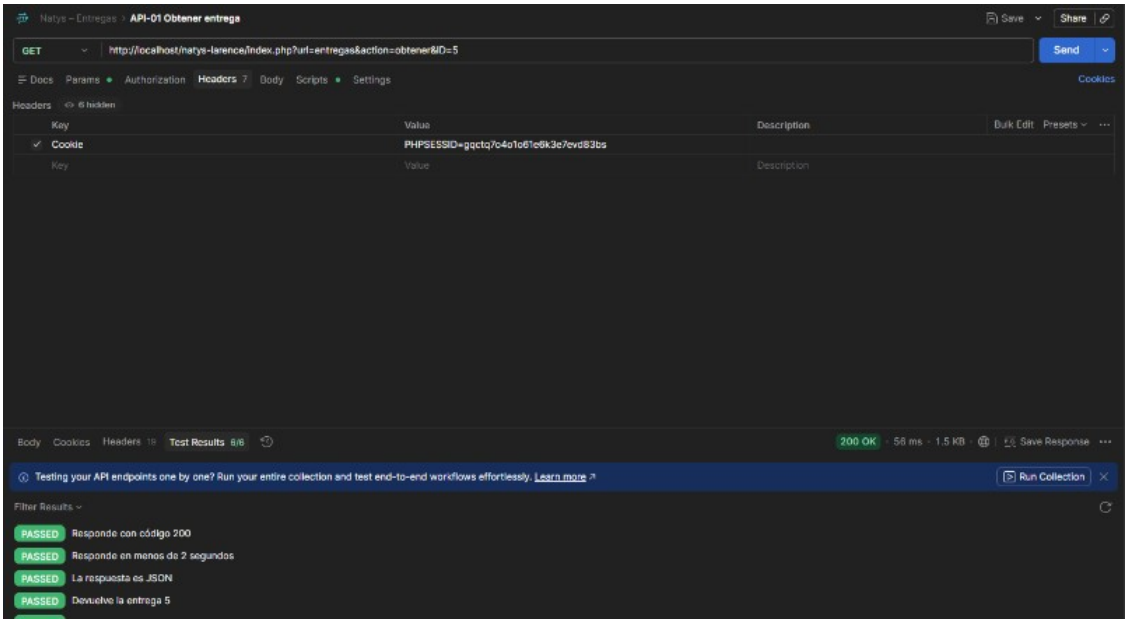
AC-01 aprobada desde tests\aceptacion

Captura 28 – Regresión con Selenium IDE



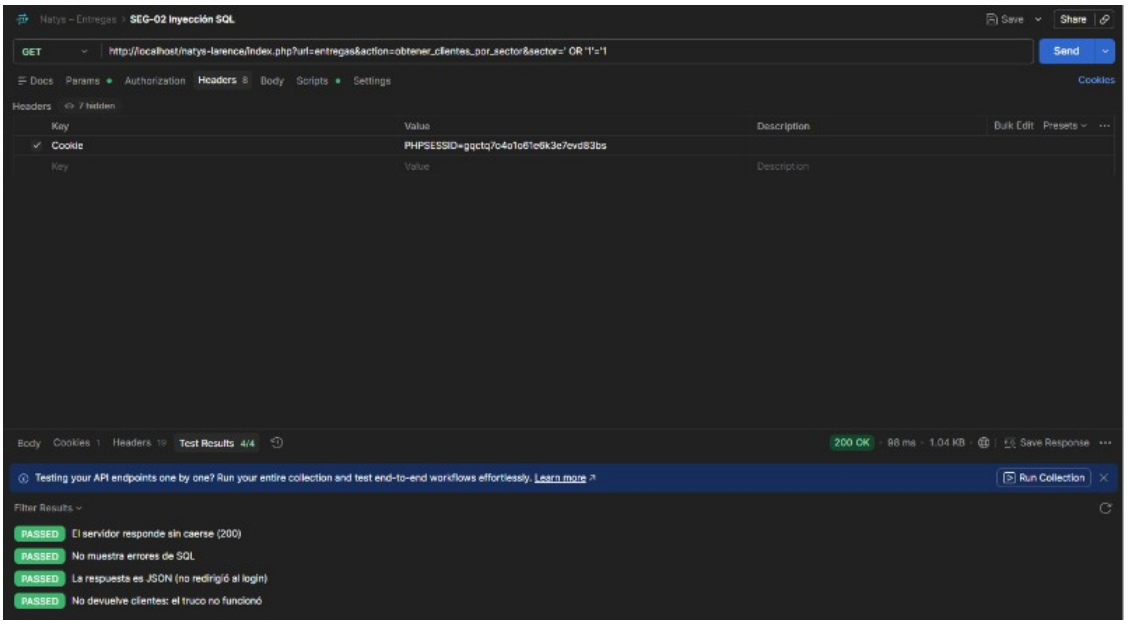
UI-01: 8 pasos en verde

Captura 29a – Regresión con Postman: API-01



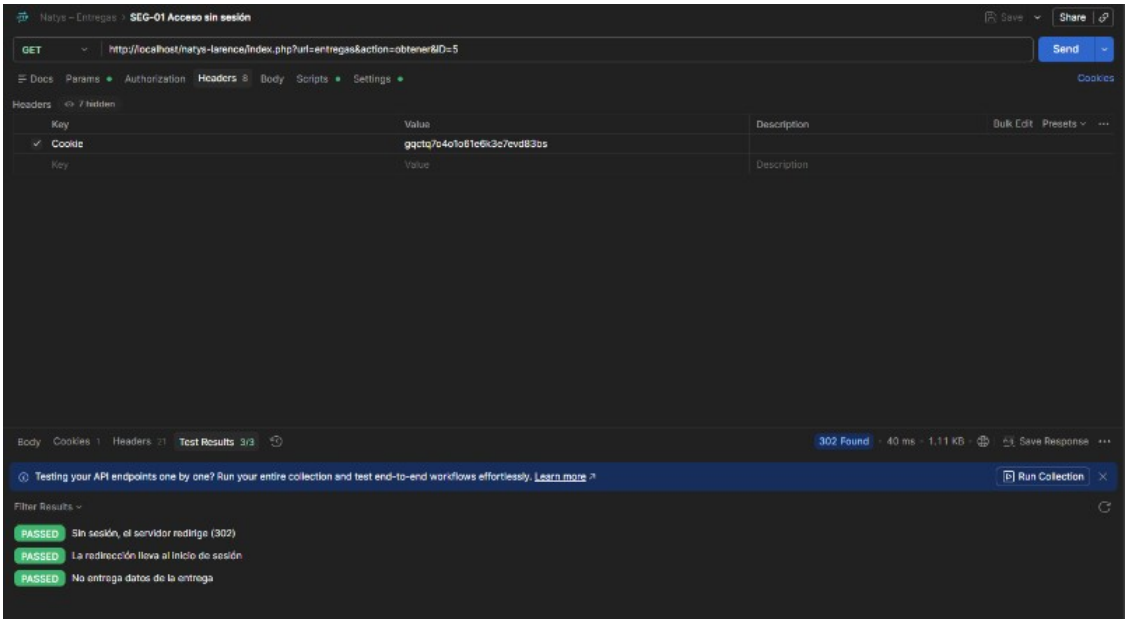
6/6 validaciones aprobadas, 200 OK

Captura 29b – Regresión con Postman: SEG-02



4/4 validaciones aprobadas, 200 OK y lista vacía

Captura 29c – Regresión con Postman: SEG-01



3/3 validaciones aprobadas, 302 Found

21. Conclusión

Las pruebas unitarias cubrieron las operaciones principales del modelo Entrega: registrar, consultar, obtener, modificar, eliminar y cancelar. La prueba UT-01 detectó que el registro aceptaba datos incompletos y reportaba éxito ante un error de la base de datos; el defecto se corrigió con una validación de campos obligatorios y un manejo correcto de excepciones, y la re-ejecución lo confirmó.

Todas las pruebas se aislaron con mocks, por lo que evalúan solo la lógica del modelo y se ejecutan en milisegundos sin XAMPP. La suite completa terminó con 7 de 7 pruebas aprobadas y 28 aserciones. La prueba de integración IT-01 complementó a las unitarias: al usar la base de datos real detectó el BUG-02 en la acción modificar, que los mocks no podían revelar. Tras corregirlo, la prueba pasó a estado aprobado. La revisión de la interfaz reveló además el BUG-03, causado por la propia corrección y cubierto ahora por la prueba de integración, y el BUG-04 en el formulario de edición, corregido en el modelo y con la fecha pendiente en el JavaScript. La prueba de backend API-01 confirmó que el endpoint obtener entrega los datos correctos en formato JSON y que rechaza las peticiones sin una sesión válida, y la prueba de humo SM-01 verificó en 0,14 segundos que el servidor, la base de datos, el modelo y la página del módulo funcionan. Finalmente, la prueba de interfaz y sistema UI-01 con Selenium IDE comprobó, como lo haría un usuario, que el formulario de edición muestra correctamente el sector y los pedidos de la entrega. Las pruebas de seguridad SEG-01 y SEG-02 confirmaron que el módulo no entrega datos sin una sesión válida y que resiste la inyección SQL gracias al uso de consultas preparadas. Por último, la prueba de rendimiento RD-01 con JMeter mostró que el módulo atiende 50 usuarios simultáneos con un tiempo promedio de 985 ms y sin errores, y la prueba de aceptación AC-01 con Codeception confirmó que el módulo cumple los requisitos RF 14.2 y RF 14.5 desde el punto de vista del usuario. La regresión final repitió todas las pruebas automatizadas con PHPUnit, Codeception, Selenium IDE y Postman, y todas resultaron aprobadas, lo que confirma que las correcciones no afectaron funcionalidades existentes.